

FLUTTER · FIREBASE · MATERIAL DE ESTUDO

Racha App

O código do app inteiro, explicado arquivo por arquivo

Pasta `lib/` · 83 arquivos · ~12.500 linhas

Versão de 05/10/2026 · Flutter 3.47 · Riverpod · Firebase

O que tem aqui

PARTE 0 — ANTES DE COMEÇAR

- 0.1 Como usar este material
- 0.2 O mapa do app
- 0.3 Conceitos que aparecem em todo lugar

PARTE 1 — ENTRADA, ROTAS E TEMA

- 1.1 main.dart
- 1.2 app_router.dart
- 1.3 app_theme.dart
- 1.4 firebase_options.dart

PARTE 2 — MODELOS

- 2.1 enums.dart
- 2.2 user_model.dart
- 2.3 grupo_model.dart
- 2.4 racha_model.dart
- 2.5 participante_model.dart
- 2.6 convidado_model.dart
- 2.7 avaliacao_model.dart
- 2.8 estatistica_model.dart
- 2.9 ranking_model.dart
- 2.10 solicitacao e aviso
- 2.11 alvo_avaliacao.dart

PARTE 3 — SERVICES

- 3.1 auth_service.dart
- 3.2 storage_service.dart
- 3.3 location_service.dart
- 3.4 geocoding_service.dart
- 3.5 notificações

PARTE 4 — REPOSITORIES

- 4.1 user_repository.dart
- 4.2 grupo_repository.dart
- 4.3 racha_repository.dart
- 4.4 participante_repository.dart
- 4.5 os demais repositories

PARTE 5 — PROVIDERS E CONTROLLERS

- 5.1 firebase_providers.dart
- 5.2 auth_controller.dart
- 5.3 perfil_controller.dart
- 5.4 criar_racha_com_admin.dart
- 5.5 racha_controller.dart
- 5.6 grupo_controller.dart
- 5.7 participante e convidado
- 5.8 solicitacao_controller.dart
- 5.9 times_controller.dart
- 5.10 avaliação e ranking
- 5.11 estatistica_controller.dart
- 5.12 notification_controller.dart

PARTE 6 — LÓGICA PURA

- 6.1 jogador_elegivel e pesos
- 6.2 setores.dart
- 6.3 balanceador_times.dart
- 6.4 agregador_ranking.dart
- 6.5 mvp_calculator.dart
- 6.6 utilitários

PARTE 7 — WIDGETS REAPROVEITADOS

- 7.1 avatar_jogador.dart
- 7.2 widgets pequenos
- 7.3 foto_perfil_picker.dart
- 7.4 foto_racha.dart

PARTE 8 — TELAS

- 8.1 login_screen.dart
- 8.2 cadastro_screen.dart
- 8.3 perfil e editar perfil
- 8.4 home_screen.dart
- 8.5 criar_racha_screen.dart
- 8.6 criar_grupo_screen.dart
- 8.7 editar_racha e grupo
- 8.8 localizacao_picker_screen.dart
- 8.9 rachas_proximos_screen.dart
- 8.10 grupo_detalhe_screen.dart
- 8.11 racha_detalhe_screen.dart
- 8.12 racha_tabs_section.dart
- 8.13 convidar_jogador_screen.dart
- 8.14 avaliacao_screen.dart
- 8.15 ranking e histórico

PARTE 9

Perguntas que o professor pode fazer

0.1 Como usar este material

Cada arquivo da pasta `lib/` tem uma seção aqui, na ordem em que faz sentido aprender: primeiro a base (modelos, banco), depois a lógica, por último as telas, que usam tudo o que veio antes.

Nada é explicado duas vezes. Quando um arquivo repete algo que já apareceu em outro, a seção mostra uma caixa cinza assim:

Já explicado em **8.1 login_screen.dart**: `Form`, `TextFormField`, `validator` e o botão com carregamento.

e só explica o que aquele arquivo tem de **novo**. Então, se a pergunta do professor cair num trecho que parece "pulado", siga o link da caixa cinza — a explicação está lá.

O código mostrado aqui são **os trechos que importam**, não o arquivo inteiro. Os comentários do próprio código (em cinza itálico) também valem como explicação — no projeto eles explicam o **porquê** de cada decisão.

Dica de estudo: leia a Parte 0 inteira primeiro. Ela explica os conceitos que aparecem em praticamente todo arquivo (`ref.watch`, `async/await`, `AsyncValue` ...), e o resto do material parte do princípio de que você já sabe o que eles são.

0.2 O mapa do app

O código é dividido em **camadas**. Cada uma tem uma responsabilidade e só conversa com a camada vizinha:

Pasta	O que faz	Conhece o Firebase?
<code>screens/</code>	As telas. Mostram dados e chamam ações.	Não (no máximo o tipo <code>GeoPoint</code>)
<code>widgets/</code> , <code>core/widgets/</code>	Pedaços de tela reaproveitados (avatar, seletor de foto).	Não
<code>providers/</code>	Liga tudo: entrega dados para as telas e executa as ações (os "controllers").	Só para criar as instâncias
<code>repositories/</code>	Lê e grava no Firestore. Uma classe por coleção.	Sim
<code>services/</code>	Fala com o mundo de fora: login, GPS, busca de endereço, notificação.	Sim
<code>models/</code>	As "fichas" de cada coisa: usuário, racha, grupo, participante...	Só tipos (<code>Timestamp</code> , <code>GeoPoint</code>)

core/	Lógica pura: balancear times, calcular ranking e MVP, tema, rotas.	Não (no máximo o tipo <code>GeoPoint</code>)
-------	--	---

O caminho de **qualquer ação** do app é sempre o mesmo. Exemplo: o jogador aperta "Confirmar presença":



A tela **não** atualiza a si mesma depois de gravar. Ela está "ouvindo" o banco (um *stream*), e quando o dado muda lá, ela se redesenha sozinha. Isso é o que faz o app mostrar a mudança no celular de todo mundo ao mesmo tempo.

Por que separar assim? Porque a lógica importante (balancear times, calcular ranking) fica em `core/`, sem nada de Firebase, e por isso dá para testar com `flutter test` sem internet e sem banco. São 62 testes, todos dessa parte.

0.3 Conceitos que aparecem em todo lugar

Estes conceitos aparecem em quase todos os arquivos. Quando um arquivo usar um deles, ele aponta para cá (C1, C2...).

C1 Widget, StatelessWidget e StatefulWidget

No Flutter, **tudo que aparece na tela é um widget**: texto, botão, coluna, a tela inteira. Todo widget tem um método `build`, que devolve o que desenhar.

- **StatelessWidget** — não guarda nada que mude. Recebe dados e desenha.
- **StatefulWidget** — tem uma classe `State` separada que guarda valores que mudam (o que foi digitado, se está carregando). Para mudar um valor e redesenhar, chama `setState(() { ... })`.

```
class _CriarRachaScreenState extends ConsumerState<CriarRachaScreen> {
  DateTime? _data; // valor que muda

  Future<void> _escolherData() async {
    final data = await showDatePicker(...);
    if (data != null) setState(() => _data = data); // muda e redesenha
  }
}
```

O `_` no começo do nome (`_data`, `_CriarRachaScreenState`) significa **privado**: só esse arquivo enxerga.

C2 const, final, late e parâmetros nomeados

- `final` — recebe valor uma vez e nunca mais muda.
- `const` — o valor é conhecido já na compilação. Num widget, `const Text('Olá')` deixa o Flutter reaproveitar o mesmo objeto em vez de criar outro a cada redesenho.
- `late` — "vou dar valor depois, antes de usar". Usado quando o valor depende de algo que só existe no `initState`.
- `{required this.nome}` — parâmetro **nomeado**: quem chama escreve `UserModel(nome: 'Ana')`. `required` obriga a passar.
- `super.key` — repassa a `key` para a classe mãe. A key ajuda o Flutter a reconhecer o mesmo widget entre redesenhos. Todo widget tem por padrão.

C3 Null safety: ?, !, ?? e ?.

- `String?` — pode ser nulo. `String` sem `?` nunca é nulo.
- `valor!` — "eu garanto que não é nulo aqui". Se for, o app quebra.
- `a ?? b` — usa `a`; se for nulo, usa `b`. Ex.: `user?.nome ?? 'Carregando...'`.
- `a?.b` — só acessa `b` se `a` não for nulo; senão o resultado é nulo.

C4 Future, async/await e Stream

- **Future** — um valor que chega **depois** (uma leitura do banco, por exemplo). `await` espera ele chegar sem travar a tela; só funciona dentro de função marcada com `async`.
- **Stream** — uma sequência de valores ao longo do tempo. O Firestore usa isso para avisar sempre que um documento muda (`.snapshots()`).
- `stream.first` — pega só o primeiro valor de um stream, como se fosse um Future. Aparece quando o código precisa ler uma vez só.
- `Future.wait([a, b])` — dispara várias leituras **ao mesmo tempo** e espera todas. Bem mais rápido que uma depois da outra.

C5 Riverpod: os providers

Riverpod é a biblioteca que **entrega dados e objetos** para quem precisa, sem passar tudo de tela em tela. Um provider é uma variável global que sabe criar um valor:

Tipo	Para quê	Exemplo no app
<code>Provider</code>	Um objeto que não muda (um repository, um service)	<code>userRepositoryProvider</code>
<code>StreamProvider</code>	Dados ao vivo do banco	<code>meusGruposProvider</code>
<code>FutureProvider</code>	Dado lido uma vez	<code>rankingDoGrupoProvider</code>
<code>.family</code>	O mesmo provider com um parâmetro	<code>participantesDoRachaProvider(rachaId)</code>
<code>AsyncNotifierProvider</code>	Um controller com ações (ver C8)	<code>authControllerProvider</code>

Para tudo isso funcionar, o app inteiro fica dentro de um `ProviderScope` (ver [1.1 main.dart](#)).

C6 ConsumerWidget, ref.watch e ref.read

Uma tela que usa providers estende `ConsumerWidget` (ou `ConsumerStatefulWidget` se tiver estado). Ela recebe um `ref`, que é a porta para os providers:

- `ref.watch(provider)` — lê e **fica observando**. Quando o valor muda, a tela redesenha. Usado dentro do `build`.
- `ref.read(provider)` — lê **uma vez**, sem observar. Usado dentro de ações (o `onPressed` de um botão).
- `ref.listen(provider, (antes, depois) {...})` — executa algo quando o valor muda, sem redesenhar.

```
final grupos = ref.watch(meusGruposProvider);           // tela observa
...
onPressed: () => ref.read(authControllerProvider.notifier).logout(), // ação
```

`.notifier` devolve o controller em si (com os métodos), em vez do estado dele.

C7 AsyncValue: carregando, erro ou dado

Quando a tela observa um `StreamProvider` ou `FutureProvider`, ela recebe um `AsyncValue`, que está sempre em um de três estados. O `.when` obriga a tratar os três:

```
return grupos.when(  
  data: (lista) => ..., // chegou  
  loading: () => const CircularProgressIndicator(), // carregando  
  error: (e, _) => Text('Erro: $e'), // deu erro  
);
```

- `maybeWhen(data: ..., orElse: ...)` — trata só o que interessa; o resto cai no `orElse`.
- `valueOrNull` — o dado, ou nulo se ainda não chegou. Usado quando não precisa mostrar carregamento.
- `.isLoading`, `.hasError`, `.error` — perguntas diretas sobre o estado.

C8 Controller com AsyncNotifier

Toda ação que grava algo passa por um **controller**. Todos seguem o mesmo molde:

```
class RachaController extends AsyncNotifier<void> {  
  @override  
  Future<void> build() async {} // estado inicial: nada  
  
  Future<void> remover(String rachaId) async {  
    state = const AsyncLoading(); // 1. avisa "carregando"  
    state = await AsyncValue.guard(() { // 2. executa  
      return ref.read(rachaRepositoryProvider).remover(rachaId);  
    }); // 3. vira dado ou erro  
  }  
}
```

`AsyncValue.guard` executa a função e **captura qualquer erro**, guardando no `state` em vez de deixar o app quebrar. A tela observa esse `state` para mostrar o botão girando (`isLoading`) ou a mensagem de erro (`error`).

C9 Navegação: go_router e Navigator

- `context.push('/rachas/abc')` — abre a tela da rota, empilhando por cima da atual.
- `context.push('/grupos/x', extra: grupo)` — `extra` passa um objeto inteiro para a próxima tela.
- `context.pop()` ou `Navigator.of(context).pop()` — fecha a tela (ou diálogo) de cima.
- `Navigator.pop(context, valor)` — fecha **devolvendo** um valor para quem abriu. É assim que os diálogos de confirmação devolvem `true` / `false`.

As rotas estão todas em [1.2 app_router.dart](#).

C10 Firestore: o banco de dados

- **Coleção** é uma pasta de **documentos**; cada documento é um mapa de campos (`{ 'nome': 'Ana', 'idade': 20 }`). Ex.: `users/{uid}` .
- **Subcoleção** é uma coleção dentro de um documento. Ex.: `rachas/{id}/participantes` — os participantes daquele racha.
- `.get()` lê uma vez (Future). `.snapshots()` fica ouvindo (Stream).
- `.where('campo', isEqualTo: x)` filtra.
- `collectionGroup('avaliacoes')` busca em **todas** as subcoleções com esse nome, de todos os rachas de uma vez.
- **WriteBatch** — junta várias gravações e manda tudo junto: ou grava tudo, ou nada. Usado sempre que uma falha no meio deixaria o banco inconsistente.

Quem decide quem pode ler e gravar cada coisa é o arquivo `firestore.rules` , fora da pasta `lib/` .

C11 Recursos do Dart 3 que aparecem bastante

- **switch como expressão** — devolve um valor direto: `final cor = switch (status) { pendente => amarelo, recusado => vermelho, ... };`
- **Records** — vários valores juntos sem criar classe: `final (rotulo, cor) = ('Confirmado', verde);`
- **if/for dentro de listas** — `[ if (isAdmin) Botao(), for (final p in lista) Linha(p) ]` .
- **Spread** `...lista` — "espalha" os itens de uma lista dentro de outra.
- **extension** — adiciona métodos a um tipo que já existe. Ex.: `TipoCampo.society.label` (ver [2.1 enums.dart](#)).

C12 mounted depois de await

Depois de um `await` , a pessoa pode ter saído da tela. Usar o `context` de uma tela que já fechou quebra o app. Por isso aparece muito:

```
await ref.read(authControllerProvider.notifier).login(...);
if (!mounted) return; // a tela ainda existe? senão, para aqui
ScaffoldMessenger.of(context).showSnackBar(...);
```

Em funções soltas (fora de um `State`) a mesma checagem é `context.mounted` .

1.1 main.dart

lib/main.dart

O ponto de partida do app: liga o Firebase, prepara as datas em português e desenha o widget raiz, `RachaApp`.

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  FirebaseMessaging.onBackgroundMessage(_aoReceberEmSegundoPlano);
  await initializeDateFormatting('pt_BR');
  runApp(const ProviderScope(child: RachaApp()));
}
```

ensureInitialized()

Precisa vir antes de qualquer `await` no `main`: o Firebase usa código nativo do celular, e essa linha liga a ponte entre Flutter e o sistema antes do app desenhar.

Firebase.initializeApp

Conecta ao projeto `racha-app-ghad-108ab`. As chaves vêm de [firebase_options.dart](#).

initializeDateFormatting('pt_BR')

Carrega os nomes de dias e meses em português. Sem isso, `DateFormat("EEEE", 'pt_BR')` mostraria "Saturday" ou daria erro.

ProviderScope

Envolve o app inteiro. É onde o Riverpod guarda os valores de todos os providers ([C5](#)). Sem ele, nenhum `ref.watch` funciona.

_aoReceberEmSegundoPlano

Função vazia exigida pelo Firebase Messaging para notificações que chegam com o app fechado. Precisa ser função de nível de topo (fora de classe) e ter `@pragma('vm:entry-point')`, porque roda num processo separado.

```
class RachaApp extends ConsumerWidget {
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final router = ref.watch(routerProvider);
    ref.watch(notificationSyncProvider);

    return MaterialApp.router(
      title: 'Racha App',
      debugShowCheckedModeBanner: false,
      theme: AppTheme.dark,
      routerConfig: router,
    );
  }
}
```

MaterialApp.router

Versão do `MaterialApp` que usa o `go_router` para navegar (em vez de `home:` com uma tela fixa).

ref.watch(notificationSyncProvider)

O valor não é usado: observar esse provider só serve para "ligá-lo", e ele registra o celular para notificações. Ver [5.12](#).

theme: AppTheme.dark

O tema escuro do app inteiro, em [1.3](#).

1.2 app_router.dart

lib/core/router/app_router.dart

A lista de todas as telas do app e o endereço de cada uma, além da regra que manda quem não está logado para o login.

```
final routerProvider = Provider<GoRouter>((ref) {
  final authState = ref.watch(authStateChangesProvider);

  return GoRouter(
    initialLocation: '/login',
    refreshListenable: GoRouterRefreshStream(ref, authStateChangesProvider),
    redirect: (context, state) {
      final logado = authState.valueOrNull != null;
      final indoParaAuth =
        state.matchedLocation == '/login' || state.matchedLocation == '/cadastro';

      if (!logado && !indoParaAuth) return '/login';
      if (logado && indoParaAuth) return '/home';
      return null;
    },
    routes: [
      GoRoute(path: '/login', builder: (context, state) => const LoginScreen()),
      GoRoute(path: '/home', builder: (context, state) => const HomeScreen()),
      GoRoute(
        path: '/rachas/:id',
        builder: (context, state) =>
          RachaDetalheScreen(rachaId: state.pathParameters['id']!),
      ),
      GoRoute(
        path: '/grupos/:id',
        builder: (context, state) =>
          GrupoDetalheScreen(grupo: state.extra as GrupoModel),
      ),
      // ... mais 12 rotas no mesmo formato
    ],
  );
});
```

redirect

Roda antes de toda navegação. Deslogado tentando abrir qualquer tela → vai para `/login`. Logado tentando abrir login ou cadastro → vai para `/home`. `return null` = "pode seguir". É **por isso que**

nenhuma tela precisa navegar depois do login: quando o Firebase avisa que alguém entrou, o redirect manda para a Home sozinho.

refreshListenable

Faz o `redirect` rodar de novo toda vez que o estado de login muda. `GoRouterRefreshStream`, no fim do arquivo, é uma pequena ponte: escuta o provider do Riverpod e avisa o `go_router`.

:id e pathParameters

`/rachas/:id` aceita qualquer valor no lugar de `:id`. A tela recebe esse valor por `state.pathParameters['id']`.

extra

Algumas rotas recebem o objeto inteiro em vez de só o id (`state.extra` as `GrupoModel`), para a tela já abrir com os dados sem precisar buscar de novo. Ver [C9](#).

Rota	Tela
<code>/login</code> , <code>/cadastro</code>	Login e cadastro
<code>/home</code>	Tela inicial
<code>/perfil</code> , <code>/perfil/editar</code>	Perfil do jogador
<code>/proximos</code>	Rachas Próximos (busca por distância)
<code>/grupos/criar</code> , <code>/rachas/criar</code>	Criar racha recorrente / avulso
<code>/grupos/:id</code> (+ <code>/ranking</code> , <code>/historico</code> , <code>/editar</code>)	Tela do grupo e suas telas filhas
<code>/rachas/:id</code> (+ <code>/convidar</code> , <code>/editar</code> , <code>/avaliar</code>)	Tela de uma rodada e suas telas filhas

1.3 app_theme.dart

lib/core/theme/app_theme.dart

As cores e o visual de todos os componentes, num lugar só. Mudar uma cor aqui muda o app inteiro.

```
class AppColors {
  AppColors._();

  static const fundo = Color(0xFF121212);
  static const superficie = Color(0xFF1E1E1E);
  static const verde = Color(0xFF1DB954);
  static const textoSecundario = Color(0xFFB3B3B3);

  // Semânticas: verde deu certo, âmbar está esperando, vermelho deu errado.
  static const confirmado = verde;
  static const pendente = Color(0xFFFFBBF2);
  static const recusado = Color(0xFFFF8717);
  static const destaque = Color(0xFFFFFC10); // troféu e estrela
}
```

AppColors._O

Construtor privado: ninguém consegue criar um `AppColors()`. A classe só existe para agrupar as constantes.

static const

Acessadas direto pela classe: `AppColors.verde`, sem criar objeto.

Color(0xFF1DB954)

Cor em hexadecimal. `FF` é a opacidade (totalmente opaca), e `1DB954` é o verde em RGB.

Duas famílias de cor

As de marca (fundo, verde) dão a cara do app. As semânticas carregam significado — a regra do projeto é nunca usar uma cor semântica só por ser bonita.

```
static ThemeData get dark {
  final base = ColorScheme.fromSeed(seedColor: AppColors.verde, brightness: Brightness.dark);
  final colorScheme = base.copyWith(
    primary: AppColors.verde,
    onPrimary: Colors.black,      // texto preto sobre o verde
    surface: AppColors.fundo,
    ...
  );
  return ThemeData(
    useMaterial3: true,
    colorScheme: colorScheme,
    scaffoldBackgroundColor: AppColors.fundo,
    cardTheme: const CardThemeData(color: AppColors.superficie, elevation: 0, ...),
    inputDecorationTheme: InputDecorationTheme(filled: true, ...),
    filledButtonTheme: FilledButtonThemeData(...),
    // ... um tema para cada tipo de componente
  );
}
```

ColorScheme.fromSeed

A partir de uma cor só, o Flutter gera as dezenas de tons que o Material 3 precisa. Depois o `copyWith` troca à mão só as que definem a identidade (o tom gerado sai mais apagado que o verde desejado).

Um tema por componente

`cardTheme`, `inputDecorationTheme`, `filledButtonTheme` ... Definindo aqui, todo card, campo e botão do app fica igual sem repetir estilo em cada tela.

get

`static ThemeData get dark` é um *getter*: usado como propriedade (`AppTheme.dark`), mas executa código.

1.4 firebase_options.dart

lib/firebase_options.dart

Arquivo **gerado automaticamente** pelo comando `flutterfire configure`. Contém os identificadores do projeto no Firebase (para Android e para a web). Não se edita à mão. Essas chaves são públicas por natureza — vão dentro de todo app instalado; quem protege os dados são as regras do Firestore.

O que é um modelo

Um modelo é a "ficha" de uma coisa do app: um usuário, um racha, um participante. É uma classe só com dados, sem tela e sem banco. Ela sabe duas coisas: **virar mapa** para ser gravada no Firestore, e **ser montada a partir de um mapa** lido de lá.

2.1 enums.dart

lib/models/enums.dart

Um `enum` é uma lista fechada de opções. Em vez de gravar `'society'` como texto solto (e alguém digitar `'Society'` errado), o código usa `TipoCampo.society`, e o compilador não deixa usar uma opção que não existe.

```
enum TipoCampo { campao, society, futsal, minicampo }

extension TipoCampoX on TipoCampo {
  String get label => switch (this) {
    TipoCampo.campao => 'Campão',
    TipoCampo.society => 'Society',
    TipoCampo.futsal => 'Futsal',
    TipoCampo.minicampo => 'Minicampo',
  };

  int get qtdJogadoresLinhaPadrao => switch (this) {
    TipoCampo.campao => 10,
    TipoCampo.futsal => 4,
    TipoCampo.society => 6,
    TipoCampo.minicampo => 7,
  };

  bool get qtdConfiguravel =>
    this == TipoCampo.society || this == TipoCampo.minicampo;
}
```

extension

Adiciona propriedades ao enum (C11). `tipo.label` dá o nome bonito para a tela;

`tipo.qtdJogadoresLinhaPadrao` preenche o campo de jogadores sozinho quando o admin escolhe o tipo de campo.

qtdConfiguravel

Campão e futsal têm formação fixa; society e minicampo deixam o admin mudar. As telas de criar racha usam isso para habilitar ou travar o campo de quantidade.

Enum	Opções	Usado para
<code>Posicao</code>	goleiro, zagueiro, lateral, volante, meia, atacante	Onde o jogador joga
<code>RachaStatus</code>	aberto, emAndamento, finalizado	Ciclo de vida da rodada
<code>TimeRacha</code>	A, B	Time sorteado pelo balanceamento

StatusConfirmacao	pendente, confirmado, recusado	O que o jogador disse que ia fazer
PresencaFinal	compareceu, atrasou, faltou	O que ele fez no dia (chamada)
StatusAprovacao	pendente, aprovado, recusado	Convidado e pedido de entrada
TipoJogador	user, convidado	Saber em qual coleção está o jogador
DiaSemana	segunda ... domingo	Dia fixo do racha recorrente

Por que StatusConfirmacao e PresencaFinal são separados? Quem recusou com antecedência e quem confirmou e não apareceu são situações bem diferentes para quem organiza — o primeiro avisou a tempo. Com um campo só, as duas ficavam iguais depois do jogo.

2.2 user_model.dart — o padrão de todo modelo

lib/models/user_model.dart

Este é o modelo explicado em detalhe. **Todos os outros modelos seguem exatamente o mesmo formato**, então nas próximas seções só aparece o que for diferente.

```
class UserModel {
  final String id;
  final String nome;
  final String email;
  final String? fotoPerfilBase64;
  final int idade;
  final double peso;
  final DateTime createdAt;
  final String? fcmToken;

  const UserModel({
    required this.id,
    required this.nome,
    required this.email,
    this.fotoPerfilBase64,
    required this.idade,
    required this.peso,
    required this.createdAt,
    this.fcmToken,
  });
}
```

Tudo final

Um modelo nunca muda depois de criado. Para "mudar", cria-se uma cópia com o valor novo (o `copyWith`, abaixo). Isso evita bugs de uma tela alterar o objeto que outra está mostrando.

String? fotoPerfilBase64

Opcional: quem não tirou foto fica com nulo (`C3`). A foto é guardada como texto base64 dentro do documento (ver [3.2](#)).

Não tem senha

De propósito: a senha fica com o Firebase Authentication, que guarda com segurança. Guardar de novo aqui seria redundante e menos seguro.

FROMMAP — DO BANCO PARA O OBJETO

```
factory UserModel.fromMap(String id, Map<String, dynamic> map) {
  return UserModel(
    id: id,
    nome: map['nome'] as String,
    email: map['email'] as String,
    fotoPerfilBase64: map['fotoPerfilBase64'] as String?,
    idade: map['idade'] as int,
    peso: (map['peso'] as num).toDouble(),
    createdAt: (map['createdAt'] as Timestamp).toDate(),
    fcmToken: map['fcmToken'] as String?,
  );
}
```

factory

Um construtor que pode ter lógica antes de criar o objeto. Aqui ele lê o mapa que veio do Firestore e monta o `UserModel`.

Map<String, dynamic>

Um documento do Firestore chega assim: chave em texto, valor de qualquer tipo. O `as String` diz qual tipo esperar.

(map['peso'] as num).toDouble()

O Firestore devolve `70` como inteiro e `70.5` como decimal. `num` aceita os dois, e o `toDouble()` padroniza.

Timestamp → DateTime

O Firestore guarda datas no tipo dele, `Timestamp`; `.toDate()` converte para o `DateTime` do Dart.

O id vem separado

O id é o nome do documento, não um campo dentro dele. Por isso é passado à parte.

TOMAP — DO OBJETO PARA O BANCO

```
String get nomeBusca => nome.toLowerCase();

Map<String, dynamic> toMap() {
  return {
    'nome': nome,
    'nomeBusca': nomeBusca,
    'email': email,
    'fotoPerfilBase64': fotoPerfilBase64,
    'idade': idade,
    'peso': peso,
    'createdAt': Timestamp.fromDate(createdAt),
    'fcmToken': fcmToken,
  };
}
```

O caminho inverso

Transforma o objeto num mapa para gravar. Não inclui o `id`, que vira o nome do documento.

nomeBusca

O nome em minúsculas, gravado só para a busca de jogadores funcionar (o Firestore não tem "contém"; ver [4.1](#)). Como sai de um getter dentro do `toMap`, nunca fica desatualizado.

COPYWITH — CÓPIA COM ALTERAÇÕES

```
UserModel copyWith({String? nome, int? idade, double? peso, String? fotoPerfilBase64, ...}) {  
  return UserModel(  
    id: id,  
    nome: nome ?? this.nome,  
    idade: idade ?? this.idade,  
    ...  
  );  
}
```

Cria um novo objeto igual ao atual, trocando só o que foi passado. O `nome ?? this.nome` significa "o novo nome, se veio; senão o antigo". Usado na edição de perfil: `atual.copyWith(nome: 'Ana')`.

Já explicado em [2.2 user_model.dart](#): campos `final`, construtor com `required`, `fromMap`, `toMap` e `copyWith`. As seções abaixo mostram só o que cada modelo tem de diferente.

2.3 grupo_model.dart

O **racha recorrente** ("Racha da Quarta"): a configuração fixa que gera uma rodada nova toda semana.

```
final DiaSemana diaSemana;
final String horario;          // "HH:mm"
final List<String> membrosFixos; // convidados toda semana
final GeoPoint? localizacao;    // ponto no mapa
final bool abertoParaNovosMembros;
final List<String> auxiliares;  // quem pode fazer chamada e conferir estatísticas
final String? fotoBase64;      // foto do campo

// no fromMap:
membrosFixos: List<String>.from(map['membrosFixos'] as List? ?? const []),
abertoParaNovosMembros: map['abertoParaNovosMembros'] as bool? ?? false,
```

Enum gravado como texto

No `toMap`: `diaSemana.name` (vira `'sabado'`). No `fromMap`: `DiaSemana.values.byName(...)` faz o caminho de volta. Todos os modelos gravam enums assim.

?? const [] e ?? false

Campos que foram criados depois que já existiam grupos no banco. Um grupo antigo não tem o campo, então o `fromMap` usa um valor padrão em vez de quebrar.

GeoPoint

Tipo do Firestore para latitude e longitude. Usado na busca por distância ([8.9](#)).

auxiliares

Os "anotadores": membros a quem o admin deu permissão de fazer a chamada e conferir estatísticas. Não é co-administração.

2.4 racha_model.dart

Uma **rodada**: um jogo numa data específica. Pode ser avulsa (`grupoId` nulo) ou ter nascido de um grupo.

```
final String? grupoId;
final DateTime dataHora;
final RachaStatus status;
final String?.mvpUserId;
final List<String> anotadores; // copiado dos auxiliares do grupo

int get totalVagas => qtdJogadoresLinha + 1;

bool podeFazerChamada(String? userId) =>
  userId != null && (userId == adminId || anotadores.contains(userId));

bool podeConferirEstatistica(String? userId, String jogadorId) =>
  podeFazerChamada(userId) && (userId == adminId || userId != jogadorId);
```

totalVagas

Jogadores de linha mais o goleiro: o tamanho de **um** time.

podeFazerChamada

Admin ou anotador. Fica no modelo porque várias telas fazem essa mesma pergunta.

podeConferirEstatistica

A mesma gente da chamada, menos o anotador nos próprios números — senão ele conferiria os próprios gols e voltaria a ser autodeclaração. O admin pode, porque já pode editar a estatística de qualquer um.

anotadores copiados do grupo

Guardar a lista no próprio racha deixa a regra de segurança do Firestore decidir a permissão lendo um documento só, em vez de dois.

2.5 participante_model.dart

Um **usuário dentro de uma rodada**. O mesmo jogador tem um participante diferente em cada racha.

```
final String userId;
final Posicao? posicaoMain;
final Posicao? posicaoUsual;
final TimeRacha? time; // nulo até gerar os times
final StatusConfirmacao statusConfirmacao;
final PresencaFinal? presenca; // nulo até a chamada

bool get confirmado => statusConfirmacao == StatusConfirmacao.confirmado;
```

posicaoMain e posicaoUsual

A principal decide o setor do jogador no balanceamento; a usual (secundária) só é usada se faltar gente em outro setor (6.2).

presenca nula

Nula numa rodada finalizada quer dizer "ninguém fez a chamada", não "faltou".

Getter confirmado

Atalho legível: `p.confirmado` em vez de comparar o enum toda vez.

2.6 convidado_model.dart

Um jogador **sem conta**, cadastrado por outro para uma rodada só. Idade e peso são aproximados. Diferenças em relação ao participante:

statusAprovacao

O admin precisa aprovar o convidado para ele entrar nos times.

oficializadoComoUserId

Se o convidado criar conta depois, o admin "oficializa": o histórico dele (notas, gols) passa para a conta nova. Este campo guarda para qual conta foi.

2.7 avaliacao_model.dart

A nota (1 a 5) que um jogador dá a outro depois do jogo.

```
final String avaliadorId;    // sempre um usuário
final String avaliadoId;    // usuário OU convidado
final TipoJogador avaliadoTipo;
final double nota;
final String? grupoId;
```

avaliadoTipo

O id sozinho não diz se é usuário ou convidado (estão em coleções diferentes). Este campo diz onde procurar.

grupoId copiado

O racha já sabe o grupo, mas copiar aqui deixa o ranking do grupo buscar todas as avaliações numa consulta só, em vez de percorrer rodada por rodada.

2.8 estatistica_model.dart

Gols, assistências e cartões de um jogador numa rodada.

```
final int gols;
final int assistencias;
final int cartoesAmarelos;
final int cartoesVermelhos;
final bool confirmada;      // alguém conferiu?
final String? conferidaPor;

// no fromMap:
confirmada: map['confirmada'] as bool? ?? true,
```

confirmada

O jogador lança os próprios números, que ficam pendentes até o admin ou um anotador conferir. Só estatística conferida entra no ranking. Sem isso, bastava digitar 10 gols para virar artilheiro.

?? true no fromMap

Estatísticas antigas, de antes da conferência existir, não têm o campo. Elas contam como conferidas para não sumir do ranking um histórico que ninguém mais tem como conferir.

2.9 ranking_model.dart

O **acumulado** de um jogador: média de notas, MVPs, gols, assistências, rodadas. Não é digitado por ninguém — é sempre **calculado** a partir das avaliações e estatísticas (6.4). O id do documento é o próprio `userId`.

2.10 `solicitacao_model.dart` e `aviso_model.dart`

SolicitacaoModel

Um usuário pedindo para entrar num grupo aberto (achado em "Rachas Próximos"). Diferente do convidado: aqui é alguém com conta, e aprovar o torna membro fixo do grupo.

AvisoModel

Um recado curto (mensagem e data) que aparece no sino da Home. Existe para contar à pessoa algo que aconteceu longe da tela dela — ex.: o grupo em que ela pediu para entrar foi apagado. Ler = apagar.

2.11 `alvo_avaliacao.dart`

Diferente dos outros: **não vai para o banco**. São duas classes só para a tela de avaliação saber quem mostrar.

AlvoAvaliacao

Alguém a ser avaliado: o id (de usuário ou de convidado), o tipo e, se for convidado, o nome.

ContextoAvaliacao

Tudo que a tela precisa: se a pessoa já avaliou, em que time ela está, a lista de companheiros e a de adversários.

Detalhe que costuma cair: o id usado na avaliação é o `userId` do jogador, nunca o id do documento de participante. O participante muda a cada rodada; se fosse ele, as notas de rodadas diferentes não se somariam no ranking.

O que é um service

Um service é uma classe que **conversa com algo de fora do app**: o login do Firebase, o GPS, um site de busca de endereço, as notificações do celular. Cada uma "embrulha" uma biblioteca externa, para o resto do app não depender dela diretamente.

3.1 auth_service.dart

lib/services/auth_service.dart

```
class AuthService {
  AuthService(this._firebaseAuth);

  final FirebaseAuth _firebaseAuth;

  Stream<User?> get authStateChanges => _firebaseAuth.authStateChanges();

  Future<UserCredential> cadastrar({required String email, required String senha}) {
    return _firebaseAuth.createUserWithEmailAndPassword(email: email, password: senha);
  }

  Future<UserCredential> login({required String email, required String senha}) {
    return _firebaseAuth.signInWithEmailAndPassword(email: email, password: senha);
  }

  Future<void> recuperarSenha({required String email}) {
    return _firebaseAuth.sendPasswordResetEmail(email: email);
  }

  Future<void> logout() => _firebaseAuth.signOut();
}
```

AuthService(this._firebaseAuth)

O `FirebaseAuth` é **recebido** pelo construtor, não criado aqui dentro. Isso se chama injeção de dependência: quem cria o service (o provider em [5.1](#)) decide qual instância passar.

authStateChanges

Um Stream ([C4](#)) que avisa toda vez que alguém entra ou sai. Nulo = ninguém logado. É ele que faz o router trocar de tela sozinho ([1.2](#)).

Os outros métodos

Só repassam para o Firebase, com nomes em português. A camada é fina de propósito: não sabe nada de Firestore nem de modelos.

3.2 storage_service.dart

lib/services/storage_service.dart

```
class StorageService {
  Future<String> uploadFotoPerfil({
    required String userId,
    required Uint8List bytes,
  }) async {
    return base64Encode(bytes);
  }
}
```

Por que base64?

O lugar certo para fotos seria o Firebase Storage, mas ele exige o plano pago. Enquanto isso, a foto é convertida em texto (base64) e gravada dentro do próprio documento do usuário.

Por que o nome "upload"?

Para o dia em que o Storage for ligado: basta trocar o corpo deste método para enviar a foto e devolver a URL. Quem chama não precisa mudar nada.

Uint8List

A foto em bytes crus. Usar bytes (e não um `File`) faz funcionar também no navegador, onde não existe caminho de arquivo.

3.3 location_service.dart

lib/services/location_service.dart

```
class LocationService {
  Future<PosicaoAtual> obterPosicaoAtual() async {
    if (!await Geolocator.isLocationServiceEnabled()) {
      throw Exception('Ative a localização (GPS) do aparelho e tente de novo.');
```

```
    }

    var permissao = await Geolocator.checkPermission();
    if (permissao == LocationPermission.denied) {
      permissao = await Geolocator.requestPermission();
    }
    if (permissao == LocationPermission.denied ||
        permissao == LocationPermission.deniedForever) {
      throw Exception('Permissão de localização negada.');
```

```
    }

    final posicao = await Geolocator.getCurrentPosition(
      locationSettings: const LocationSettings(accuracy: LocationAccuracy.high),
    );
    return PosicaoAtual(
      ponto: GeoPoint(posicao.latitude, posicao.longitude),
      precisaoMetros: posicao.accuracy,
    );
  }
}
```

A ordem das checagens

GPS ligado? → tem permissão? se não, pede → negou? lança erro. Só então lê a posição. A tela que chama mostra a mensagem do `Exception` para a pessoa.

throw Exception(...)

Interrompe a função e manda o erro para quem chamou, que trata com `try/catch`.

LocationAccuracy.high

Liga o GPS de verdade. Mais lento, mas acerta a rua em vez da região.

PosicaoAtual

Classe do mesmo arquivo que guarda o ponto **e a precisão** em metros. No computador o navegador estima a posição pelo Wi-Fi e erra por quilômetros; com a precisão, a tela consegue avisar isso (`imprecisa` = mais de 1 km).

3.4 geocoding_service.dart

lib/services/geocoding_service.dart

Transforma um **endereço ou CEP em coordenadas**, para o admin achar o campo no mapa sem precisar arrastar. Usa dois serviços gratuitos pela internet (pacote `http`):



```
Future<List<EnderecoEncontrado>> buscar(String termo) async {
  final limpo = termo.trim();
  if (limpo.length < 3) return const [];

  final digitos = limpo.replaceAll(RegExp(r'\D'), '');
  if (digitos.length == 8 && RegExp(r'^[\d\s-]+$').hasMatch(limpo)) {
    final endereco = await _enderecoDoCep(digitos);
    if (endereco != null) {
      final achados = await _coordenadasDe(endereco);
      if (achados.isNotEmpty) return achados;
    }
  }
  return _coordenadasDe(limpo);
}
```

Como sabe que é CEP

`RegExp(r'\D')` casa tudo que **não** é dígito; trocando por vazio, sobram só os números. Oito dígitos (com ou sem hífen) = CEP.

Por que dois serviços

O ViaCEP devolve rua e bairro, mas não coordenadas. O Nominatim devolve coordenadas a partir do texto.

jsonDecode

Nos métodos privados, a resposta vem em JSON (texto) e `jsonDecode` transforma em mapas e listas do Dart.

countrycodes: 'br'

Limita a busca ao Brasil — sem isso, "Rua das Flores" trazia Portugal primeiro.

Busca só ao confirmar

O Nominatim pede no máximo uma consulta por segundo, então a tela só busca quando a pessoa aperta enter, nunca a cada letra.

3.5 notification_service.dart e local_notification_service.dart

lib/services/

NotificationService (push)

Pede permissão de notificação e pega o **token** do celular — o "endereço" para onde uma notificação seria enviada. O token é gravado no usuário. **Nada dispara push ainda**: isso exigiria Cloud Functions, que é plano pago. Na web devolve nulo.

LocalNotificationService (lembrete)

Não depende de servidor: o próprio celular agenda um lembrete **2 horas antes** do racha para quem confirmou presença, e cancela se a pessoa recusar.

```
Future<void> agendarLembrete(RachaModel racha) async {
  final horarioLembrete = racha.dataHora.subtract(_antecedencia);
  if (horarioLembrete.isBefore(DateTime.now())) return;

  await _plugin.zonedSchedule(
    id: _idNotificacao(racha.id),
    title: 'Racha chegando',
    body: '${racha.nome} começa às ${DateFormat('HH:mm').format(racha.dataHora)}',
    scheduledDate: tz.TZDateTime.from(horarioLembrete, tz.local),
    ...
  );
}

int _idNotificacao(String rachaId) => rachaId.hashCode & 0x7fffffff;
```

Se já passou, não agenda

Não faz sentido avisar de algo que já aconteceu.

Fuso horário (tz)

O agendamento precisa do fuso do aparelho; o `inicializar()` descobre qual é.

_idNotificacao

O Android exige um número inteiro como id. O `hashCode` do id do racha vira esse número — o mesmo racha sempre dá o mesmo número, e é assim que dá para cancelar depois. O `& 0x7fffffff` garante que seja positivo e caiba em 32 bits.

O que é um repository

Um repository é a **única porta para uma coleção do banco**. Toda leitura e gravação de usuários passa pelo `UserRepository`; de rachas, pelo `RachaRepository`, e assim por diante. A tela nunca escreve `FirebaseFirestore` — ela pede ao controller, que pede ao repository.

4.1 `user_repository.dart` — o padrão de todo repository

`lib/repositories/user_repository.dart`

```
class UserRepository {
  UserRepository(this._firestore);

  final FirebaseFirestore _firestore;

  CollectionReference<Map<String, dynamic>> get _collection =>
    _firestore.collection(FirestorePaths.users);

  Future<void> criar(UserModel user) {
    return _collection.doc(user.id).set(user.toMap());
  }

  Future<UserModel?> buscarPorId(String id) async {
    final doc = await _collection.doc(id).get();
    if (!doc.exists) return null;
    return UserModel.fromMap(doc.id, doc.data()!);
  }

  Stream<UserModel?> observar(String id) {
    return _collection.doc(id).snapshots().map((doc) {
      if (!doc.exists) return null;
      return UserModel.fromMap(doc.id, doc.data()!);
    });
  }

  Future<void> atualizar(UserModel user) {
    return _collection.doc(user.id).update(user.toMap());
  }
}
```

`_collection`

Atalho para a coleção `users`. O nome vem de `FirestorePaths` (6.6), para não espalhar textos soltos pelo código.

`set` x `update`

`set` grava o documento inteiro (cria se não existe). `update` muda só os campos passados e falha se o documento não existir.

`doc(user.id)`

O documento do usuário tem como nome o mesmo id do login (uid). Assim, achar o perfil de quem está logado é direto.

get() x snapshots()

`buscarPorId` lê uma vez; `observar` fica ouvindo e entrega um `UserModel` novo a cada mudança ([C10](#)). O `.map` converte cada documento em modelo com o `fromMap`.

A BUSCA DE JOGADORES POR NOME

```
Future<List<UserModel>> buscarPorNomeOuEmail(String termo) async {
  final termoBusca = termo.trim().toLowerCase();
  final limiteSuperior = termoBusca + String.fromCharCode(0xF8FF);
  final resultados = await Future.wait([
    _collection.where('email', isEqualTo: termoBusca).limit(10).get(),
    _collection
      .where('nomeBusca', isGreaterThanOrEqualTo: termoBusca)
      .where('nomeBusca', isLessThan: limiteSuperior)
      .limit(10)
      .get(),
  ]);
  // junta as duas listas sem repetir ninguém (Set "vistos")
}
```

O truque do "começa com"

O Firestore não tem "contém" nem "começa com". Mas buscar tudo entre `"gus"` e `"gus" + o maior caractere possível` pega exatamente os nomes que começam com "gus". É por isso que existe o campo `nomeBusca` em minúsculas.

Duas buscas ao mesmo tempo

Não dá para saber se a pessoa digitou nome ou email, então busca pelos dois em paralelo (`Future.wait`, [C4](#)) e junta.

`buscarVarios` busca vários usuários de uma vez, também em paralelo — usado na geração de times, que precisa do cadastro de todos os confirmados.

Já explicado em **4.1 user_repository.dart**: construtor que recebe o Firestore, `_collection`, `set`, `update`, `get`, `snapshots` e a conversão com `fromMap`. Abaixo só o que é novo.

4.2 grupo_repository.dart

```
Future<String> criar(GrupoModel grupo) async {
  final docRef = _collection.doc();
  await docRef.set(grupo.toMap());
  return docRef.id;
}

Stream<List<GrupoModel>> observarPorMembro(String userId) {
  return _collection
    .where('membrosFixos', arrayContains: userId)
    .snapshots()
    .map((snap) => snap.docs.map((d) => GrupoModel.fromMap(d.id, d.data())).toList());
}

Future<void> atualizarFoto(String groupId, String? fotoBase64) {
  return _collection.doc(groupId).update({
    'fotoBase64': fotoBase64 ?? FieldValue.delete(),
  });
}

Future<void> sair(String groupId, String userId) {
  return _collection.doc(groupId).update({
    'membrosFixos': FieldValue.arrayRemove([userId]),
  });
}
```

`_collection.doc()` sem id

O Firestore gera um id aleatório. O método devolve esse id para quem chamou poder criar a primeira rodada do grupo.

Consultas que devolvem lista

`snap.docs` são todos os documentos encontrados; cada um vira um modelo.

`arrayContains`

"Grupos em que a lista `membrosFixos` contém este usuário" — é como a Home acha os grupos em que a pessoa participa.

`FieldValue.delete()`

Apaga o campo do documento (usado para remover a foto).

`FieldValue.arrayRemove`

Tira um item da lista **direto no servidor**, sem regravar a lista inteira — se o admin adicionou alguém no mesmo segundo, essa adição não se perde.

4.3 racha_repository.dart

```
DocumentReference<Map<String, dynamic>> novoDocRef() => _collection.doc();

void criarEmLote(WriteBatch batch, DocumentReference<Map<String, dynamic>> ref, RachaModel racha) {
  batch.set(ref, racha.toMap());
}

Stream<RachaModel?> observarAtualPorGrupo(String groupId) {
  return _collection
    .where('groupId', isEqualTo: groupId)
    .where('status', isEqualTo: RachaStatus.aberto.name)
    .orderBy('dataHora')
    .limit(1)
    .snapshots()
    .map((snap) => snap.docs.isEmpty ? null : RachaModel.fromMap(...));
}
```

Métodos "EmLote"

Não gravam na hora: só colocam a gravação dentro de um `WriteBatch` ([C10](#)). Quem chamou faz o `commit()` no fim, e tudo é gravado junto.

novoDocRef

Reserva um id antes de gravar. Necessário no batch: para convidar os participantes da rodada nova, é preciso saber o id dela antes de ela existir.

observarAtualPorGrupo

"A rodada aberta mais próxima deste grupo": filtra por grupo e status, ordena por data, pega só a primeira. Consultas com mais de um filtro precisam de um índice no Firestore (`firestore.indexes.json`).

atualizar

Faz `update` só dos campos do formulário de edição, de propósito: o `status` e o MVP têm fluxo próprio e não podem ser sobrescritos por quem só editou o horário.

4.4 participante_repository.dart

```
CollectionReference<Map<String, dynamic>> _collection(String rachaId) =>
    _firestore
        .collection(FirestorePaths.rachas)
        .doc(rachaId)
        .collection(FirestorePaths.participantes);

Stream<List<ParticipanteModel>> observarMeusConvites(String userId) {
    return _firestore
        .collectionGroup(FirestorePaths.participantes)
        .where('userId', isEqualTo: userId)
        .snapshots()
        .map(...);
}

Future<void> convidar({required String rachaId, required String userId, ...}) async {
    final existente = await buscarPorUserId(rachaId, userId);
    if (existente != null) return;
    await _collection(rachaId).add(participante.toMap());
}
```

Subcoleção

`_collection` agora é uma **função** que recebe o racha: os participantes ficam em `rachas/{id}/participantes`.

collectionGroup

Procura em **todas** as subcoleções `participantes` de todos os rachas. É assim que a Home descobre todas as rodadas para as quais a pessoa foi convidada.

Sem convite duplicado

Antes de convidar, verifica se a pessoa já está no racha.

add x set

`add` cria um documento com id gerado automaticamente, como o `doc()` + `set` do grupo.

Os demais métodos (`atualizarStatus`, `registrarPresenca`, `definirPosicoes`, `definirTimeEmLote`) são `update` de um ou dois campos.

4.5 Os demais repositories

Arquivo	O que tem de novo
<code>convidado_repository</code>	Mesmo formato do de participante (subcoleção <code>convidados</code>). Nada novo.
<code>avaliacao_repository</code>	<code>avaliarEmLote</code> : grava todas as notas de uma pessoa num batch. Se falhasse no meio, ela ficaria marcada como "já avaliou" com metade das notas. <code>reatribuirParaUser</code> : troca o avaliado de convidado para a conta nova (oficialização).
<code>estatistica_repository</code>	O id do documento é o próprio <code>jogadorId</code> , então salvar de novo substitui em vez de duplicar. <code>confirmar</code> : marca várias estatísticas como conferidas num batch.
<code>ranking_repository</code>	<code>buscarVarios</code> em paralelo e <code>salvar</code> com <code>set</code> .

<code>solicitacao_repository</code>	Pedidos de entrada (<code>grupos/{id}/solicitacoes</code>). Ordena no próprio app em vez de no banco, para não exigir índice por causa de uma lista de 1 ou 2 itens.
<code>aviso_repository</code>	Recados em <code>users/{id}/avisos</code> . <code>dispensar</code> apaga o documento.

A cola entre telas e banco

Esta pasta tem dois tipos de coisa: **providers de dados**, que as telas observam para mostrar informação, e **controllers**, que executam as ações (criar, confirmar, finalizar...). Os conceitos de Riverpod estão em [C5](#)–[C8](#).

5.1 `firebase_providers.dart`

`lib/providers/firebase_providers.dart`

PARTE 1: CRIANDO OS OBJETOS (INJEÇÃO DE DEPENDÊNCIA)

```
final firebaseAuthProvider = Provider<FirebaseAuth>((ref) => FirebaseAuth.instance);
final firestoreProvider = Provider<Firestore>((ref) => FirebaseFirestore.instance);

final authServiceProvider = Provider<AuthService>((ref) {
  return AuthService(ref.watch(firebaseAuthProvider));
});

final userRepositoryProvider = Provider<UserRepository>((ref) {
  return UserRepository(ref.watch(firestoreProvider));
});
// ... um provider para cada service e repository
```

É aqui que cada service e repository é criado, recebendo o Firebase que precisa. É o **único lugar** onde `FirebaseAuth.instance` e `FirebaseFirestore.instance` aparecem. Qualquer parte do app que precisar do repository de usuários faz `ref.read(userRepositoryProvider)`.

PARTE 2: QUEM ESTÁ LOGADO

```
final authStateChangesProvider = StreamProvider<User?>((ref) {
  return ref.watch(authServiceProvider).authStateChanges;
});

final uidLogadoProvider = Provider<String?>((ref) {
  return ref.watch(authStateChangesProvider).value?.uid;
});
```

`authStateChangesProvider`

Transforma o stream de login ([3.1](#)) num provider. O router e várias telas observam ele.

`uidLogadoProvider` — importante

O id de quem está logado. **Todo provider que lê o banco observa este.** Motivo: quando alguém sai e outra pessoa entra, os providers precisam ser recriados. Sem isso, a leitura da conta anterior dava erro de permissão durante o login, e a tela ficava travada nesse erro mesmo depois de a nova conta entrar.

PARTE 3: OS DADOS QUE AS TELAS OBSERVAM

```
final meusGruposProvider = StreamProvider<List<GrupoModel>>((ref) {
  final uid = ref.watch(authStateChangesProvider).value?.uid;
  if (uid == null) return Stream.value(const []);
  return ref.watch(grupoRepositoryProvider).observarPorAdmin(uid);
});

final participantesDoRachaProvider =
  StreamProvider.family<List<ParticipanteModel>, String>((ref, rachaId) {
    if (ref.watch(uidLogadoProvider) == null) return Stream.value(const []);
    return ref.watch(participanteRepositoryProvider).observarPorRacha(rachaId);
  });
```

Deslogado = lista vazia

`Stream.value(const [])` é um stream que entrega uma lista vazia e acaba. Evita consultar o banco sem ninguém logado (as regras recusariam).

.family

`participantesDoRachaProvider('abc')` e `participantesDoRachaProvider('xyz')` são dois providers separados, um por racha. O Riverpod guarda cada um e reaproveita se outra tela pedir o mesmo.

O arquivo tem uns 25 providers nesse formato. Os que fazem algo a mais:

Provider	O que faz além de repassar
<code>gruposQueParticipoProvider</code>	Grupos em que a pessoa é membro, tirando os que ela mesma administra (esses já aparecem em "Meus rachas").
<code>proximasRodadasProvider</code>	Pega todos os convites, busca o racha de cada um em paralelo e descarta os que já passaram (com 1h de tolerância), os finalizados e os que a pessoa organiza. Ordena do mais próximo ao mais distante.
<code>fichaDoJogadorProvider</code>	Para o admin decidir um pedido de entrada: nota média, MVPs e a posição em que a pessoa mais joga.
<code>contextoAvaliacaoProvider</code>	Monta o <code>ContextoAvaliacao</code> (2.11): em que time a pessoa está, companheiros e adversários. Só conta quem continua confirmado.
<code>rankingDoGrupoProvider</code>	Busca rodadas, avaliações e estatísticas do grupo (3 consultas em paralelo) e passa tudo para <code>agregarRanking</code> (6.4).
<code>historicoDoGrupoProvider</code>	Só as rodadas finalizadas, da mais recente para a mais antiga.

5.2 auth_controller.dart — o padrão de todo controller

lib/providers/auth_controller.dart

Já explicado em C8: o molde `AsyncNotifier` com `state = AsyncLoading()` e `AsyncValue.guard`. Aqui o foco é o que o cadastro faz.

```
Future<void> cadastrar({
  required String nome, required String email, required String senha,
  required int idade, required double peso, Uint8List? fotoPerfil,
}) async {
  state = const AsyncLoading();
  state = await AsyncValue.guard(() async {
    try {
      final emailNormalizado = email.trim().toLowerCase();
      final credential =
        await authService.cadastrar(email: emailNormalizado, senha: senha);
      final uid = credential.user!.uid;

      String? fotoBase64;
      if (fotoPerfil != null) {
        fotoBase64 = await storageService.uploadFotoPerfil(userId: uid, bytes: fotoPerfil);
      }

      final user = UserModel(id: uid, nome: nome, email: emailNormalizado,
        fotoPerfilBase64: fotoBase64, idade: idade, peso: peso,
        createdAt: DateTime.now());
      await userRepository.criar(user);
    } on FirebaseAuthException {
      throw Exception('Não foi possível criar a conta. Verifique os dados e tente novamente.');
```

Três passos em ordem

1) cria a conta no Firebase Auth (devolve o `uid`); 2) converte a foto; 3) grava o documento do usuário no Firestore com o mesmo `uid` como id.

Email em minúsculas

A busca de jogadores compara em minúsculas; sem isso, "Fulano@Gmail.com" nunca seria encontrado.

Mensagem de erro genérica

Se o email já tem conta, o Firebase diz isso. O app esconde e mostra uma mensagem genérica, para ninguém usar o cadastro para descobrir quem tem conta no app.

on FirebaseAuthException

Captura só esse tipo de erro e troca por outro. Qualquer outro erro passa direto e cai no `guard`.

AuthController.new

Forma curta de passar o construtor da classe para o provider criar o controller.

login e **logout** só chamam o service. **recuperarSenha** engole o erro de propósito: a tela sempre diz "se existir uma conta, o link foi enviado", exista ou não — mesmo cuidado do cadastro.

Depois do login ninguém navega: o `authStateChanges` muda e o redirect do router ([1.2](#)) leva para a Home.

Daqui em diante, todo controller segue o molde acima (`state = AsyncLoading()` + `AsyncValue.guard` + chamar um ou mais repositories). Só as regras de negócio de cada um são explicadas.

5.3 `perfil_controller.dart`

Busca o usuário atual, troca a foto se veio uma nova, e grava com `copyWith` ([2.2](#)) só nome, idade, peso e foto. Email e senha não mudam aqui: são do Firebase Auth.

5.4 `criar_racha_com_admin.dart`

```
Future<String> criarRachaComAdmin(Ref ref, RachaModel racha) async {
  final rachaRepo = ref.read(rachaRepositoryProvider);
  final batch = ref.read(firestoreProvider).batch();

  final docRef = rachaRepo.novoDocRef();
  rachaRepo.criarEmLote(batch, docRef, racha);
  ref.read(participanteRepositoryProvider).convidarEmLote(
    batch, rachaId: docRef.id, userId: racha.adminId);

  await batch.commit();
  return docRef.id;
}
```

Função compartilhada

Criar racha avulso e criar grupo faziam a mesma coisa (gravar o racha e convidar o admin). Ficou num lugar só.

Batch

Ou nasce o racha com o admin convidado, ou não nasce nada.

O admin entra como pendente

Quem organiza nem sempre joga, então confirma presença como todo mundo.

5.5 `racha_controller.dart`

criar, atualizar e remover só repassam. O que tem regra:

FINALIZAR — ENCERRA A RODADA E CRIA A PRÓXIMA

```
Future<void> finalizar(RachaModel racha) async {
  state = const AsyncLoading();
  state = await AsyncValue.guard(() async {
    final estatisticas = await ref.read(estatisticaRepositoryProvider)
      .observarPorRacha(racha.id).first;
    final pendentes = estatisticas.where((e) => !e.confirmada).length;
    if (pendentes > 0) {
      throw Exception('$pendentes estatística(s) ainda esperando conferência...');
    }

    final batch = ref.read(firestoreProvider).batch();
    rachaRepo.atualizarStatusEmLote(batch, rachaId: racha.id, status: RachaStatus.finalizado);

    final grupoId = racha.grupoId;
    if (grupoId != null) {
      final grupo = await ref.read(grupoRepositoryProvider).buscarPorId(grupoId);
      if (grupo != null) {
        final proximaRef = rachaRepo.novoDocRef();
        final proxima = RachaModel(
          id: proximaRef.id,
          grupoId: grupo.id,
          dataHora: proximaOcorrencia(grupo.diaSemana, grupo.horario),
          anotadores: grupo.auxiliares,
          ...
        );
        rachaRepo.criarEmLote(batch, proximaRef, proxima);
        for (final userId in {grupo.adminId, ...grupo.membrosFixos}) {
          participanteRepo.convidarEmLote(batch, rachaId: proximaRef.id, userId: userId);
        }
      }
    }
    await batch.commit();
  });
}
```

Primeiro: estatísticas conferidas

Não finaliza se alguma estatística ainda espera conferência — depois de finalizada ela ficaria de fora do ranking sem ninguém perceber.

Racha de grupo gera o próximo

Lê a configuração do grupo, calcula a próxima data (`proximaOcorrencia` , [6.6](#)), cria a rodada e convida o admin e todos os membros fixos.

{grupo.adminId, ...grupo.membrosFixos}

Chaves criam um `Set` , que não aceita repetidos: se o admin também estiver na lista de membros, ele não é convidado duas vezes.

Tudo num batch

Encerrar uma e criar a outra acontecem juntos. Se caísse no meio, o grupo poderia ficar sem nenhuma rodada aberta.

ENTRARCOMCODIGO

O "código" do racha é o próprio id do documento. Quem digita o código entra já **confirmado** (foi a própria pessoa que quis entrar). Bloqueia racha finalizado e não duplica quem já está lá. Devolve o id para a tela abrir o racha.

5.6 grupo_controller.dart

Método	Regra
<code>criar</code>	Grava o grupo e já cria a primeira rodada com <code>criarRachaComAdmin</code> (5.4).
<code>removerMembroFixo</code>	Tira da lista de membros e, por padrão, também da rodada aberta. Com <code>removerDaRodadaAberta: false</code> , a pessoa joga a rodada que já confirmou.
<code>alternarAnotador</code>	Liga/desliga a prancheta: grava no grupo (próximas rodadas) e na rodada aberta, senão só valeria na semana seguinte.
<code>sair</code>	O próprio membro saindo (usa <code>arrayRemove</code> , 4.2).
<code>remover</code>	Apaga o grupo e os pedidos de entrada, e deixa um aviso para quem esperava resposta — tudo num batch.
<code>atualizarFoto</code>	Troca ou remove a foto do racha.

5.7 participante_controller.dart e convidado_controller.dart

O de participante só repassa (convidar, mudar status, registrar presença, definir posições, remover). No de convidado, o método com regra é **oficializar**:

```
await ref.read(avaliacaoRepositoryProvider)
  .reatribuirParaUser(convidadoId: convidadoId, userId: userId);
await ref.read(estatisticaRepositoryProvider)
  .copiarParaUser(convidadoId: convidadoId, userId: userId);
await ref.read(convidadoRepositoryProvider).marcarOficializado(...);
await ref.read(rankingControllerProvider.notifier).recalcularRanking(userId);
```

Quando um convidado cria conta, as notas que ele recebeu passam para a conta nova, as estatísticas são copiadas, e o ranking é recalculado com esse histórico.

5.8 solicitacao_controller.dart

```
enum ResultadoSolicitacao { enviada, jaEraMembro, jaSolicitou, recusadoAntes, erro }

Future<ResultadoSolicitacao> solicitar(GrupoModel grupo) async {
  ...
  if (grupo.membrosFixos.contains(uid)) { resultado = ResultadoSolicitacao.jaEraMembro; return; }
  final ultima = await repo.buscarUltimaSolicitacao(grupo.id, uid);
  if (ultima?.status == StatusAprovacao.pendente) { resultado = ...jaSolicitou; return; }
  if (ultima?.status == StatusAprovacao.recusado) { resultado = ...recusadoAntes; return; }
  await repo.criar(SolicitacaoModel(...));
  await _avisar(userId: grupo.adminId, mensagem: '... pediu para entrar no ${grupo.nome}.');
}
```

Devolve um resultado

Diferente dos outros controllers, este diz **o que aconteceu**, para a tela mostrar a mensagem certa

("você já tem um pedido pendente", "você já é membro"...).

Recusa não expira

Quem foi recusado não consegue pedir de novo sozinho. Só o admin reabre (`reabrir`), seja um dia ou um ano depois.

aprovar

Torna a pessoa membro fixo e, se houver rodada aberta, já convida para ela.

`_avisar engole erro`

O aviso é extra. Se o pedido foi gravado e só o aviso falhou, melhor manter o pedido do que dizer que ele não foi enviado.

5.9 `times_controller.dart`

Prepara os dados e chama o algoritmo de balanceamento ([6.3](#)), depois grava o time de cada um.

```
final participantes = (await participanteRepo.observePorRacha(racha.id).first)
    .where((p) => p.confirmando).toList();
final convidados = (await convidadoRepo.observePorRacha(racha.id).first)
    .where((c) => c.aprovado).toList();

final users = await userRepo.buscarVarios(userIds);
final rankings = await ref.read(rankingRepositoryProvider).buscarVarios(userIds);

for (final p in participantes) {
    elegiveis.add(JogadorElegivel(
        id: p.id, tipo: TipoJogador.user, nome: user.nome,
        posicaoMain: p.posicaoMain, posicaoUsual: p.posicaoUsual,
        nota: _notaDe(rankings[p.userId]),
        idade: user.idade, peso: user.peso,
        golsPorJogo: _golsPorJogoDe(rankings[p.userId]),
    ));
}
// convidados entram com nota neutra (3,0) e idade/peso aproximados

final resultado = const BalanceadorTimes()
    .gerar(elegiveis, qtdJogadoresLinha: racha.qtdJogadoresLinha);
// grava time A/B de todo mundo num batch
```

Só quem conta

Participantes confirmados e convidados aprovados.

Junta dois tipos num só

Participante e convidado viram `JogadorElegivel` ([6.1](#)): o algoritmo trata os dois igual.

`_notaDe`

A média real de avaliação do jogador. Média zero quer dizer "nunca foi avaliado", então usa a nota neutra (3,0) em vez de tratá-lo como o pior do racha.

`_golsPorJogoDe`

Total de gols dividido pelas rodadas. Sem histórico devolve nulo, e o algoritmo trata como a média do grupo. No fim, um batch grava o time de todo mundo: ou todos ganham time, ou ninguém.

5.10 `avaliacao_controller.dart` e `ranking_controller.dart`

AvaliacaoController.enviarAvaliacoes grava todas as notas num batch (com o `grupoId` copiado do racha) e depois recalcula o ranking de cada usuário avaliado. Convidados não têm ranking.

```
Future<void> recalcularRanking(String userId) async {
  final avaliacoes = await ref.read(avaliacaoRepositoryProvider).buscarRecebidasPor(userId);
  final estatisticas = await ref.read(estadisticaRepositoryProvider).buscarTodasDe(userId);
  final rachasComMvp = await ref.read(rachaRepositoryProvider).buscarPorMvp(userId);

  final agregado = agregarRanking(
    avaliacoes: avaliacoes,
    estatisticas: estatisticas,
    mvpUserIds: [for (var i = 0; i < rachasComMvp.length; i++) userId],
  );

  await ref.read(rankingRepositoryProvider)
    .salvar(agregado[userId] ?? RankingModel(userId: userId));
}
```

Recalcula do zero, sempre

Em vez de somar "+1 gol" no valor anterior, busca tudo que o jogador já recebeu e refaz a conta. Assim, se um número ficou errado por qualquer motivo, a próxima vez conserta.

A conta em si

Fica em `agregarRanking` (6.4), a mesma usada pelo ranking do grupo.

calcularMvpDaRodada

Usa `calcularMvpDoRacha` (6.5), grava o MVP no racha e recalcula o ranking de quem ganhou — e de quem perdeu, se o MVP mudou.

5.11 `estatistica_controller.dart`

```
Future<void> salvar({required RachaModel racha, required String jogadorId, ...}) async {
  final uid = ref.read(firebaseAuthProvider).currentUser!.uid;
  final conferida = racha.podeConferirEstatistica(uid, jogadorId);

  await ref.read(estadisticaRepositoryProvider).salvar(EstatisticaModel(
    ...
    confirmada: conferida,
    conferidaPor: conferida ? uid : null,
  ));

  if (jogadorTipo == TipoJogador.user) {
    await ref.read(rankingControllerProvider.notifier).recalcularRanking(jogadorId);
  }
}
```

Quem lança decide o status

Se quem salvou pode conferir (admin ou anotador lançando por outro), já nasce conferida. Se foi o próprio jogador, fica pendente.

Editar volta para pendente

Toda gravação recalcula o status, então alterar um número já conferido o devolve para "aguardando conferência".

confirmar

Marca várias pendentes como conferidas de uma vez (botão "Confirmar todas") e recalcula o ranking dos usuários afetados. Ignora as que a pessoa não pode conferir.

A mesma regra está no `firestore.rules`, para ninguém burlar gravando direto no banco sem passar pelo app.

5.12 notification_controller.dart

```
final notificationSyncProvider = Provider<void>((ref) {  
  ref.read(localNotificationServiceProvider).inicializar();  
  
  ref.listen(authStateChangesProvider, (previous, next) {  
    final uid = next.valueOrNull?.uid;  
    if (uid != null) registrarToken(uid);  
  });  
  
  final tokenRefreshSub = ref.read(notificationServiceProvider)  
    .onTokenRefresh.listen((token) { ... atualizarFcmToken ... });  
  ref.onDispose(tokenRefreshSub.cancel);  
});
```

Provider<void>

Não entrega valor nenhum: existe só pelo que faz ao ser criado. O `main.dart` o observa para ligá-lo ([1.1](#)).

ref.listen

Toda vez que alguém loga, registra o token do celular no usuário ([C6](#)).

ref.onDispose

Cancela as escutas quando o provider é destruído, para não ficar consumindo memória.

try/catch com debugPrint

Falhar ao registrar notificação nunca pode travar o login.

O cérebro do app

Estes arquivos não sabem nada de tela nem de banco: recebem dados, devolvem um resultado. Por isso são os que têm testes automáticos (`test/core/`). É aqui que estão as regras mais "inteligentes" do app: montar times equilibrados, calcular ranking e eleger o MVP.

6.1 `jogador_elegivel.dart` e `balanceamento_constants.dart`

`lib/core/balanceamento/` e `lib/core/constants/`

```
class JogadorElegivel {
  final String id;           // id do participante ou do convidado
  final TipoJogador tipo;
  final Posicao? posicaoMain;
  final Posicao? posicaoUsual;
  final double nota;
  final int idade;
  final double peso;
  final double? golsPorJogo; // nulo = "não sabemos"

  bool get isGoleiro => posicaoMain == Posicao.goleiro;
}
```

A forma única com que o algoritmo enxerga qualquer jogador, venha de um participante ou de um convidado. **golsPorJogo nulo é diferente de zero**: zero é "joga e não marca"; nulo é "não sabemos".

```
const double notaNeutra = 3.0;

const double pesoNota = 1.0;
const double pesoGols = 0.6;
const double pesoIdade = 0.4;
const double pesoCorporal = 0.15;

const double escalaNota = 1.0; // 1 ponto de nota já é muita diferença
const double escalaGols = 1.0; // 1 gol por rodada
const double escalaIdade = 10.0; // 10 anos
const double escalaPeso = 15.0; // 15 kg
```

notaNeutra

Nota de quem nunca foi avaliado e de todo convidado. 3 é o meio da escala de 1 a 5: não beneficia nem prejudica.

Pesos

O quanto cada critério importa para o equilíbrio. A nota é o principal (1,0); peso corporal só desempata (0,15). **Zerar um peso desliga o critério**.

Escalas

Cada critério tem uma unidade diferente. Dividindo pela escala, "1 ponto de nota" e "10 anos de idade" passam a valer a mesma coisa na conta, e só o peso decide qual importa mais. Sem isso, o peso em kg dominaria só por ser um número maior.

6.2 setores.dart

lib/core/balanceamento/setores.dart

Num racha, zagueiro e lateral são intercambiáveis. O algoritmo pensa em **três setores**, não em seis posições:

```
enum SetorCampo { defesa, meio, ataque }

SetorCampo? setorDe(Posicao? posicao) => switch (posicao) {
  Posicao.zagueiro || Posicao.lateral => SetorCampo.defesa,
  Posicao.volante || Posicao.meia => SetorCampo.meio,
  Posicao.atacante => SetorCampo.ataque,
  Posicao.goleiro => null,
  null => null,
};

Map<SetorCampo, int> distribuicaoIdeal(int qtdJogadoresLinha) {
  const tabela = <int, (int, int, int)>{
    4: (2, 1, 1),    // futsal
    6: (2, 2, 2),    // society
    10: (4, 4, 2),   // campão
    ...
  };
}
```

Goleiro devolve nulo

Goleiro tem tratamento próprio no balanceador (é a única vaga obrigatória).

distribuicaoIdeal

Quantos jogadores cada setor deveria ter, pela formação. Os valores são *records* ([C11](#)) de três números: defesa, meio, ataque. Acima de 10 usa uma proporção.

DISTRIBUIR POR SETOR — A REGRA DA POSIÇÃO SECUNDÁRIA

Coloca cada jogador no setor da posição **principal**. Depois, onde faltar gente (comparando com o alvo):

1. Primeiro entram os que **não escolheram posição** — não tiram ninguém do lugar.
2. Se ainda faltar, puxa de um setor que está **sobrando** alguém cuja posição **secundária** é o setor em falta. Entre os candidatos, sai o de **menor nota**: os melhores continuam onde jogam melhor.
3. Se ninguém tem esse setor como secundária, o time joga desfalcado ali. Forçar alguém a jogar fora das duas posições dele seria pior.

Exemplo: confirmaram dez atacantes e nenhum meia. Os atacantes que marcaram "também jogo de meia" descem para o meio. Os que só jogam na frente ficam lá.

6.3 balanceador_times.dart — como os times são montados

lib/core/balanceamento/balanceador_times.dart

Três garantias, **nesta ordem de prioridade**:

1. **Goleiro**: no máximo um por time.
2. **Setor**: os dois times com a mesma quantidade de defesa, meio e ataque (diferença máxima de 1).
3. **Força**: os dois times o mais parecidos possível em nota, gols, idade e peso.

PASSO 1 E 2: GOLEIROS E SETORES

```
final goleiros = elegiveis.where((j) => j.isGoleiro).toList()..sort(_porNotaEEmpate);
if (goleiros.isNotEmpty) timeA.add(goleiros.first);
if (goleiros.length > 1) timeB.add(goleiros[1]);

final linha = [...elegiveis.where((j) => !j.isGoleiro), ...goleiros.skip(2)];
final escalacao = distribuirPorSetor(linha, qtdJogadoresLinha: qtdJogadoresLinha);

for (final setor in SetorCampo.values) {
  final doSetor = escalacao.porSetor[setor]!..sort(_porNotaEEmpate);
  for (final jogador in doSetor) {
    final destino = _escolherTime(timeA, timeB, noSetorA[setor] ?? 0, noSetorB[setor] ?? 0);
    destino.add(jogador);
    ...
  }
}
```

Goleiros

Ordena por nota; o melhor vai para o A, o segundo para o B. Do terceiro em diante viram jogadores de linha (`skip(2)` pula os dois primeiros).

..sort

Os dois pontos ("cascata") ordenam a lista e devolvem a própria lista, permitindo fazer isso na mesma linha da criação.

Do melhor para o pior

Dentro de cada setor, os jogadores entram em ordem de nota, cada um no time escolhido por

`_escolherTime`:

```
List<JogadorElegivel> _escolherTime(timeA, timeB, int noSetorA, int noSetorB) {
  if (noSetorA != noSetorB) return noSetorA < noSetorB ? timeA : timeB;
  if (timeA.length != timeB.length) return timeA.length < timeB.length ? timeA : timeB;
  return _somaNota(timeA) <= _somaNota(timeB) ? timeA : timeB;
}
```

Vai para o time com **menos gente naquele setor**; empatou, o com **menos gente no total**; empatou de novo, o com **menos nota somada**. Isso já espalha os melhores: o melhor vai para A, o segundo para B, o terceiro para o time mais fraco...

PASSO 3: O AJUSTE FINO COM QUATRO CRITÉRIOS

Depois da distribuição, o algoritmo testa **todas as trocas possíveis** entre um jogador do time A e um do time B **do mesmo setor**, e faz a que mais aproxima os times. Repete até nenhuma troca melhorar.

```
double _desequilibrio(_Somas a, int tamA, _Somas b, int tamB) {
    double dif(double somaA, double somaB, double escala) =>
        (somaA / tamA - somaB / tamB).abs() / escala;

    return pesoNota * dif(a.nota, b.nota, escalaNota) +
        pesoGols * dif(a.gols, b.gols, escalaGols) +
        pesoIdade * dif(a.idade, b.idade, escalaIdade) +
        pesoCorporal * dif(a.peso, b.peso, escalaPeso);
}
```

Um número só

O desequilíbrio é a soma das diferenças das **médias** de cada critério, cada uma dividida pela escala e multiplicada pelo peso. Zero = perfeitamente equilibrado.

Média, não soma

Os times podem ter um jogador de diferença. Somas iguais, nesse caso, significariam o time menor mais forte por jogador.

Só troca dentro do mesmo setor

Goleiro por goleiro, meia por meia. Por isso o passo 3 nunca desfaz os passos 1 e 2.

.abs()

Valor absoluto: tanto faz qual time é mais forte, só importa o tamanho da diferença.

```
Map<String, double> _golsEfetivos(List<JogadorElegivel> todos) {
    final conhecidos = [for (final j in todos) if (j.golsPorJogo != null) j.golsPorJogo!];
    final media = conhecidos.isEmpty ? 0.0 : conhecidos.reduce((a, b) => a + b) / conhecidos.length;
    return {for (final j in todos) j.id: j.golsPorJogo ?? media};
}
```

Quem não tem histórico de gols entra com a **média do grupo**, não com zero. Com zero, o algoritmo juntaria todos os novatos de um lado para "compensar" o artilheiro do outro.

Exemplo para explicar na hora: dois times com a mesma nota média, mas um tem os quatro jogadores de 18 anos e o outro os de 40. O desequilíbrio de idade é alto. O algoritmo procura dois jogadores do mesmo setor, um novo no A e um velho no B, e troca. A nota continua igual, e a idade média se aproxima.

_Somas, no fim do arquivo, guarda as somas de cada critério de um time. Assim, testar uma troca é só "tira a contribuição de um e põe a do outro", sem recalculando o time inteiro — importante porque num campo são centenas de trocas testadas por rodada.

Como se sabe que funciona: os testes em `test/core/balanceamento/` montam casos (todos os novos de um lado, os artilheiros juntos) e conferem o resultado. Zerando o peso da idade ou dos gols, exatamente o teste correspondente falha.

6.4 agregador_ranking.dart

lib/core/ranking/agregador_ranking.dart

A **única** conta de ranking do app. É usada tanto para o ranking geral do jogador (Perfil e balanceamento) quanto para o ranking de um grupo, para os números nunca saírem diferentes de uma tela para outra.

```
Map<String, RankingModel> agregarRanking({
  required List<AvaliacaoModel> avaliacoes,
  required List<EstatisticaModel> estatisticas,
  required List<String>.mvpUserIds,
}) {
  final notasPorUser = <String, List<double>>{};
  final golsPorUser = <String, int>{};

  for (final avaliacao in avaliacoes) {
    if (avaliacao.avaliadoTipo != TipoJogador.user) continue;
    notasPorUser.putIfAbsent(avaliacao.avaliadoId, () => []).add(avaliacao.nota);
  }

  for (final estatistica in estatisticas) {
    if (estatistica.jogadorTipo != TipoJogador.user) continue;
    if (!estatistica.confirmada) continue;
    golsPorUser.update(estatistica.jogadorId,
      (total) => total + estatistica.gols, ifAbsent: () => estatistica.gols);
  }
  ...
  return { for (final userId in userIds) userId: RankingModel(...) };
}
```

Função pura

Recebe listas, devolve um mapa `userId → RankingModel`. Não sabe de onde vieram os dados: quem chama decide se passa tudo do jogador ou só o de um grupo.

continue

Pula para o próximo item do `for`. Aqui pula convidados (não têm ranking) e estatísticas ainda não conferidas.

putIfAbsent

"Se esse jogador ainda não tem lista, cria uma vazia" — e então adiciona a nota.

update(..., ifAbsent: ...)

"Soma os gols ao total; se ainda não tem total, começa com esses gols."

Média de zero notas

A função `_media` devolve o para lista vazia em vez de dividir por zero.

```
List<RankingModel> ordenarRanking(Iterable<RankingModel> ranking) {
  return ranking.toList()
    ..sort((a, b) {
      final porMvp = b.totalMvps.compareTo(a.totalMvps);
      if (porMvp != 0) return porMvp;
      return b.mediaAvaliacoes.compareTo(a.mediaAvaliacoes);
    });
}
```

Ordem do ranking: **mais MVPs primeiro**; empate, a melhor média. O `compareTo` devolve negativo, zero ou positivo; usar `b` antes de `a` inverte a ordem (maior primeiro).

6.5 mvp_calculator.dart

lib/core/avaliacao/mvp_calculator.dart

```
MvpResultado? calcularMvpDoRacha(List<AvaliacaoModel> avaliacoes) {
  final porAvaliado = <String, List<AvaliacaoModel>>{};
  for (final avaliacao in avaliacoes) {
    if (avaliacao.avaliadoTipo != TipoJogador.user) continue;
    porAvaliado.putIfAbsent(avaliacao.avaliadoId, () => []).add(avaliacao);
  }
  if (porAvaliado.isEmpty) return null;
  // calcula a média de cada um e fica com o melhor
}

bool _melhorQue(MvpResultado a, MvpResultado b) {
  if (a.mediaNota != b.mediaNota) return a.mediaNota > b.mediaNota;
  if (a.totalAvaliacoes != b.totalAvaliacoes) return a.totalAvaliacoes > b.totalAvaliacoes;
  return a.avaliadoId.compareTo(b.avaliadoId) < 0;
}
```

MVP = **maior média** de notas na rodada. Empate: quem **recebeu mais avaliações**. Empate de novo: ordem do id, só para o resultado ser sempre o mesmo. **Convidado nunca é MVP** (não tem ranking para guardar o título).

6.6 Utilitários

Arquivo	O que faz
<code>data_utils.dart</code>	<code>proximaOcorrencia(dia, "HH:mm")</code> : a próxima data que cai naquele dia da semana. Se hoje é o dia mas o horário passou, pula uma semana. Usa <code>% 7</code> para dar a volta na semana.
<code>geo_utils.dart</code>	<code>distanciaKm</code> : distância entre dois pontos do mapa pela fórmula de haversine (considera a curvatura da Terra, raio 6.371 km). <code>distanciaLegivel</code> : "350 m" ou "4,9 km".
<code>imagem_base64.dart</code>	<code>bytesDaFoto</code> : converte a foto base64 em bytes com cache de até 64 fotos. Sem cache, cada redesenho decodificaria de novo e a foto piscaria. Base64 corrompido devolve nulo em vez de quebrar a tela.
<code>posicao_utils.dart</code>	<code>posicaoMaisFrequente</code> : a posição em que o jogador mais jogou, para a ficha do pedido de entrada. Devolve um <i>record</i> (<code>posicao</code> , <code>vezes</code>) .

<code>string_utils.dart</code>	<code>capitalizarNome</code> : "gustavo dias" → "Gustavo Dias".
<code>firestore_paths.dart</code>	Os nomes das coleções (<code>'users'</code> , <code>'rachas'</code> ...) como constantes, para não espalhar texto solto pelo código.

Pedaços de tela usados em vários lugares

Quando o mesmo visual aparece em várias telas, ele vira um widget próprio. Mudou aqui, muda em todo lugar.

7.1 `avatar_jogador.dart`

`lib/core/widgets/avatar_jogador.dart`

```
class AvatarJogador extends StatelessWidget {
  const AvatarJogador({super.key, required this.nome, this.fotoBase64,
    this.raio = 20, this.convidado = false});

  @override
  Widget build(BuildContext context) {
    final bytes = bytesDaFoto(fotoBase64);
    if (bytes != null) {
      return CircleAvatar(radius: raio, backgroundImage: MemoryImage(bytes));
    }
    return CircleAvatar(
      radius: raio,
      backgroundColor: convidado ? AppColors.superficieAlta : AppColors.verde.withValues(alpha: 0.18),
      child: Text(_iniciais(nome), style: TextStyle(fontSize: raio * 0.72, ...)),
    );
  }
}
```

Foto ou iniciais

Com foto: `CircleAvatar` com a imagem. Sem foto: as iniciais ("Gustavo Dias" → "GD"), que ajudam a distinguir as pessoas numa lista.

MemoryImage

Desenha uma imagem que está em memória (bytes), e não num arquivo ou na internet.

Parâmetros com valor padrão

`this.raio = 20`: se quem usa não passar o tamanho, fica 20.

Convidado com outra cor

Para não confundir com um jogador com conta que só não pôs foto.

fontSize: raio * 0.72

A letra cresce junto com o círculo.

AvatarDoUsuario, no mesmo arquivo, é a versão para quem só tem o id: ela busca o usuário com `userPorIdProvider` e repassa para o `AvatarJogador`.

7.2 Widgets pequenos

Widget	O que é
--------	---------

<code>TituloSecao</code>	Título de bloco ("MEUS RACHAS"): caixa alta, letra menor, espaçada e cinza. Diferencia rótulo de conteúdo sem competir com ele. Aceita um botão opcional à direita (<code>?acao</code> só inclui o widget se ele não for nulo).
<code>InfoTile</code>	Ícone + rótulo pequeno + valor (Local, Quando...). Feito com <code>Row</code> em vez de <code>ListTile</code> para ocupar metade da altura.
<code>CampoDeToque</code>	Parece um campo de texto, mas abre um seletor ao tocar (data, horário, mapa). Usa <code>InputDecorator</code> para ter o mesmo visual dos outros campos, com rótulo e mensagem de erro.
<code>confirmar(...)</code>	Diálogo padrão de "tem certeza?". Devolve <code>true</code> só se a pessoa tocar em confirmar; fechar por fora devolve <code>false</code> (o <code>?? false</code> no fim). <code>destrutivo: true</code> pinta o botão de vermelho.

7.3 foto_perfil_picker.dart — a câmera

lib/widgets/foto_perfil_picker.dart

Requisito de hardware da disciplina: é este arquivo que usa a **câmera do celular**.

```
class FotoPerfilPicker extends StatefulWidget {
  const FotoPerfilPicker({super.key, required this.onFotoSelecionada, this.fotoAtualBase64});

  final ValueChanged<Uint8List> onFotoSelecionada;
  final String? fotoAtualBase64;
  ...
}

class _FotoPerfilPickerState extends State<FotoPerfilPicker> {
  Uint8List? _nova;

  Future<void> _tirarFoto() async {
    final foto = await ImagePicker().pickImage(
      source: ImageSource.camera,
      imageQuality: 80,
      maxWidth: 1024,
    );
    if (foto == null) return;

    final bytes = await foto.readAsBytes();
    if (!mounted) return;
    setState(() => _nova = bytes);
    widget.onFotoSelecionada(bytes);
  }
}
```

ImageSource.camera

Abre a câmera do aparelho (não a galeria). `pickImage` devolve nulo se a pessoa cancelar.

imageQuality e maxWidth

Comprime e reduz a foto: ela é gravada dentro do documento do usuário, que tem limite de 1 MB.

readAsBytes

Lê a foto como bytes. Funciona no celular e no navegador.

ValueChanged<Uint8List> onFotoSelecionada

Um **callback**: a tela que usa o picker passa uma função, e o picker a chama com a foto. Assim o picker não precisa saber se está no cadastro ou na edição de perfil.

widget.onFotoSelecionada

Dentro do `State`, os parâmetros do widget são acessados por `widget.`.

__nova ?? foto salva

No `build`, mostra a foto recém-tirada; se não tem, a já salva; se não tem nenhuma, um ícone.

7.4 foto_racha.dart — a foto do campo

lib/widgets/foto_racha.dart

Já explicado em 7.3: `pickImage`, compressão e `readAsBytes`.

```
Future<FotoEscolhida?> escolherFotoDoRacha(BuildContext context, {required bool temFoto}) async {
  final origem = await showModalBottomSheet<Object>(
    context: context,
    builder: (sheetContext) => SafeArea(
      child: Column(mainAxisSize: MainAxisSize.min, children: [
        ListTile(title: const Text('Tirar foto'),
          onTap: () => Navigator.pop(sheetContext, ImageSource.camera)),
        ListTile(title: const Text('Escolher da galeria'),
          onTap: () => Navigator.pop(sheetContext, ImageSource.gallery)),
        if (temFoto)
          ListTile(title: const Text('Remover foto'),
            onTap: () => Navigator.pop(sheetContext, 'remover')),
      ]),
    ),
  );
  if (origem == null) return null;
  if (origem == 'remover') return const FotoEscolhida.remover();
  ...
  if (base64.length > _tetoBase64) { /* avisa que ficou grande */ return null; }
  return FotoEscolhida(base64);
}
```

showModalBottomSheet

O menu que sobe de baixo da tela. Cada opção fecha o menu devolvendo um valor ([C9](#)).

Aqui a galeria é permitida

A foto do campo normalmente já está no celular. A de perfil é só câmera por causa do requisito.

FotoEscolhida.remover()

Um construtor com nome. Separa "cancelou" (nulo, não mexe em nada) de "quer remover a foto". Sem essa diferença, fechar o menu por engano apagaria a foto.

Teto de 350 KB

Esse documento é lido em toda lista de grupos; uma foto grande deixaria a tela inicial lenta para todos.

O arquivo também tem dois widgets: **MiniaturaDoRacha** (quadrado da Home; sem foto, mostra uma bola) e **CapaDoRacha** (faixa larga no topo do grupo, com botão de editar para o admin; sem foto e sem ser admin, não ocupa espaço).

As telas

As telas usam tudo das partes anteriores. A primeira, o login, é explicada em detalhe porque **quase toda tela com formulário segue o mesmo formato**. Daí em diante, só o que for novo.

8.1 login_screen.dart — a base de toda tela com formulário

lib/screens/auth/login_screen.dart

A CLASSE E O ESTADO

```
class LoginScreen extends ConsumerStatefulWidget {
  const LoginScreen({super.key});

  @override
  ConsumerState<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends ConsumerState<LoginScreen> {
  final _formKey = GlobalKey<FormState>();
  final _emailController = TextEditingController();
  final _senhaController = TextEditingController();

  @override
  void dispose() {
    _emailController.dispose();
    _senhaController.dispose();
    super.dispose();
  }
}
```

ConsumerStatefulWidget

Tela com estado (C1) que também usa providers (C6). O `ref` fica disponível em qualquer método do `State`.

createState

Liga o widget à classe que guarda o estado. O widget pode ser recriado várias vezes; o `State` sobrevive.

TextEditingController

Guarda o texto digitado num campo. `_emailController.text` lê o que está escrito.

GlobalKey<FormState>

Uma "alça" para o formulário inteiro, para poder mandar validar todos os campos de uma vez.

dispose

Roda quando a tela é fechada de vez. Os controllers usam memória e precisam ser liberados aqui; o `super.dispose()` vem por último.

A AÇÃO DE ENTRAR

```
Future<void> _submeter() async {
  if (!_formKey.currentState!.validate()) return;

  await ref.read(authControllerProvider.notifier).login(
    email: _emailController.text.trim(),
    senha: _senhaController.text,
  );

  final erro = ref.read(authControllerProvider).error;
  if (erro != null && mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Email ou senha inválidos')),
    );
  }
}
```

validate()

Roda o `validator` de cada campo. Se algum devolver uma mensagem, ela aparece embaixo do campo e a função para.

ref.read(...notifier).login

Chama o controller ([5.2](#)). A tela não fala com o Firebase.

Lê o erro depois

O controller guarda o erro no próprio estado ([C8](#)); a tela confere e mostra um aviso.

SnackBar

A barrinha de aviso que aparece embaixo e some sozinha. `ScaffoldMessenger.of(context)` é quem sabe mostrá-la.

Sucesso não faz nada

Quando o login dá certo, o router leva para a Home sozinho ([1.2](#)).

mounted

Ver [C12](#).

O BUILD

```
@override
Widget build(BuildContext context) {
  final state = ref.watch(authControllerProvider);
  final carregando = state.isLoading;

  return Scaffold(
    body: SafeArea(
      child: Center(
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(24),
          child: Form(
            key: _formKey,
            child: Column(
              mainAxisAlignment: MainAxisAlignment.stretch,
              mainAxisSize: MainAxisSize.min,
              children: [
                Icon(Icons.sports_soccer, size: 64, color: Theme.of(context).colorScheme.primary),
                TextFormField(
                  controller: _emailController,
                  keyboardType: TextInputType.emailAddress,
                  decoration: const InputDecoration(labelText: 'Email'),
                  validator: (v) => (v == null || !v.contains('@')) ? 'Email inválido' : null,
                ),
                const SizedBox(height: 12),
                TextFormField(
                  controller: _senhaController,
                  obscureText: true,
                  decoration: const InputDecoration(labelText: 'Senha'),
                  validator: (v) => (v == null || v.isEmpty) ? 'Informe a senha' : null,
                ),
                const SizedBox(height: 24),
                FilledButton(
                  onPressed: carregando ? null : _submeter,
                  child: carregando
                    ? const SizedBox(height: 20, width: 20,
                        child: CircularProgressIndicator(strokeWidth: 2))
                    : const Text('Entrar'),
                ),
                TextButton(
                  onPressed: () => context.push('/cadastro'),
                  child: const Text('Não tenho conta – cadastrar'),
                ),
              ],
            ),
          ),
        ),
      ),
    ),
  );
}
```

Widget	Para que serve
<code>Scaffold</code>	A estrutura de uma tela: fundo, espaço para a barra de cima (<code>appBar</code>), o corpo (<code>body</code>) e o botão flutuante.
<code>SafeArea</code>	Afasta o conteúdo do entalhe da câmera e da barra de status do celular.
<code>SingleChildScrollView</code>	Deixa rolar quando o teclado sobe e o conteúdo não cabe. Sem ele, dá erro de "overflow" (faixa amarela e preta).

<code>Column</code>	Empilha os filhos na vertical. <code>stretch</code> estica todos na largura toda; <code>MainAxisSize.min</code> ocupa só a altura necessária (para o <code>Center</code> centralizar).
<code>Form</code> + <code>TextFormField</code>	Campo de texto que participa da validação do formulário.
<code>validator</code>	Função que devolve <code>null</code> (tudo certo) ou o texto do erro.
<code>obscureText</code>	Esconde o que é digitado (senha).
<code> SizedBox(height: 12)</code>	Espaço em branco entre os campos.

Botão com carregamento

`onPressed: null` desativa o botão. Enquanto o login roda, ele fica desativado e mostra a bolinha girando no lugar do texto — impede clicar duas vezes. **Esse padrão se repete em todos os formulários do app.**

`ref.watch` no build

Observa o controller: quando ele passa a "carregando" e volta, a tela redesenha o botão sozinha.

`Theme.of(context)`

Pega as cores do tema ([1.3](#)).

`_recuperarSenha` abre um `AlertDialog` com um campo de email, espera o resultado do `showDialog` (o email ou nulo, se cancelou) e chama o controller. A mensagem final é sempre a mesma, exista a conta ou não.

8.2 cadastro_screen.dart

Já explicado em **8.1 login_screen.dart**: estrutura da tela, controllers e `dispose`, `Form` e `validator`, botão com carregamento e erro por `SnackBar`.

O que é novo:

```

Uint8List? _fotoPerfil;
...
FotoPerfilPicker(onFotoSelecionada: (bytes) => _fotoPerfil = bytes),
...
Row(children: [
  Expanded(child: TextFormField(controller: _idadeController, ...)),
  const SizedBox(width: 12),
  Expanded(child: TextFormField(controller: _pesoController, ...)),
]),
...
await ref.read(authControllerProvider.notifier).cadastrar(
  nome: capitalizarNome(_nomeController.text),
  idade: int.parse(_idadeController.text),
  peso: double.parse(_pesoController.text.replaceAll(',', '.')),
  fotoPerfil: _fotoPerfil,
  ...
);

```

A câmera

O `FotoPerfilPicker` (7.3) devolve a foto pelo callback, e a tela guarda em `_fotoPerfil` para mandar junto no cadastro.

Row + Expanded

Idade e peso lado a lado. `Expanded` faz cada campo ocupar metade do espaço que sobra.

int.parse / double.parse

O campo devolve texto; isso converte para número. O `replaceAll(',', '.')` aceita "72,5" (vírgula brasileira).

tryParse no validator

Na validação usa-se `int.tryParse`, que devolve nulo em vez de quebrar quando o texto não é número.

Senha com 6+ caracteres

Mínimo exigido pelo Firebase Auth.

8.3 perfil_screen.dart e editar_perfil_screen.dart

Já explicado em **8.2**: o formulário de editar perfil é o do cadastro sem email e senha.

Campos já preenchidos

`late final _nomeController = TextEditingController(text: widget.user.nome);` — o `late` é necessário porque `widget` só existe depois que o `State` é criado.

Ao salvar, volta

`context.pop()` fecha a tela de edição.

PerfilScreen

Um `ConsumerWidget` sem estado: observa `currentUserModelProvider` e, com `.when` (C7), mostra foto, dados (`InfoTile`) e o resumo do ranking (nota média, MVPs, gols, assistências).

Passando o usuário para a edição

`context.push('/perfil/editar', extra: user)` (C9).

DateFormat('dd/MM/yyyy', 'pt_BR')

Formata a data de cadastro no padrão brasileiro.

8.4 home_screen.dart

`lib/screens/home/home_screen.dart`

A tela inicial: "Meus rachas", "Rachas avulsos", "Rachas que participo" e "Próximas rodadas", mais o sino e o botão de criar.

```
final grupos = ref.watch(meusGruposProvider);
final proximasRodadas = ref.watch(proximasRodadasProvider);
final avisos = ref.watch(meusAvisosProvider).valueOrNull ?? const [];
final pendentes = proximasRodadas.valueOrNull
  ?.where((r) => r.participacao.statusConfirmacao == StatusConfirmacao.pendente)
  .length ?? 0;
final naCaixa = pendentes + avisos.length;

...
IconButton(
  icon: Badge(
    label: Text('$naCaixa'),
    isLabelVisible: naCaixa > 0,
    child: const Icon(Icons.notifications_outlined),
  ),
  onPressed: naCaixa == 0 ? null : () => _mostrarCaixaDeEntrada(context, meuUid),
),
```

O sino

Conta convites ainda sem resposta (só de rodadas que vão acontecer) mais avisos. `Badge` é a bolinha com o número em cima do ícone.

Várias seções num ListView

Cada seção é um `grupos.when(...)`; se a lista estiver vazia, devolve `SizedBox.shrink()` (um widget de tamanho zero) e a seção some.

Ao tocar num grupo

`context.push('/grupos/${grupo.id}', extra: grupo)`.

actions da AppBar

Os ícones do canto: Rachas Próximos, sino, a própria foto (leva ao perfil) e sair (com `confirmar(...)`, 7.2).

WIDGETS E FUNÇÕES DO ARQUIVO

Nome	O que é
<code>_RachaTile</code>	O cartão de um racha: miniatura da foto, nome, local/horário e uma etiqueta com o tipo de campo. <code>Material</code> + <code>InkWell</code> dão o fundo arredondado e o efeito de toque.
<code>_RodadaTile</code>	Cartão de uma rodada para a qual a pessoa foi convidada, com a cor do status (Responder / Confirmado / Recusado). Usa um <code>switch</code> que devolve um record <code>(rotulo, cor)</code> (C11).
<code>_mostrarCaixaDeEntrada</code>	O menu que sobe ao tocar no sino. Usa um <code>Consumer</code> dentro do menu para ele se redesenhar sozinho quando um aviso é dispensado.
<code>_mostrarOpcoesCriarRacha</code>	O botão flutuante "Criar racha" abre um menu: avulso, recorrente ou entrar com código.
<code>_mostrarDialogoEntrarComCodigo</code>	Diálogo com <code>StatefulBuilder</code> (ver abaixo) que chama <code>entrarComCodigo</code> (5.5) e abre o racha.

StatefulBuilder em diálogos: um diálogo não tem `setState` próprio. O `StatefulBuilder` dá um `setState` só para o diálogo (para mostrar "carregando" ou o erro). As variáveis precisam ser declaradas **fora** do builder: ele roda de novo a cada `setState`, e uma variável declarada dentro voltaria ao valor inicial.

8.5 criar_racha_screen.dart

Já explicado em [8.1](#): formulário, validação, botão com carregamento.

```
TipoCampo _tipoCampo = TipoCampo.campao;
DateTime? _data;
TimeOfDay? _horario;

@override
void initState() {
  super.initState();
  _qtdController = TextEditingController(text: _tipoCampo.qtdJogadoresLinhaPadrao.toString());
}

void _onTipoCampoChanged(TipoCampo? tipo) {
  if (tipo == null) return;
  setState(() {
    _tipoCampo = tipo;
    _qtdController.text = tipo.qtdJogadoresLinhaPadrao.toString();
  });
}
```

initState

Roda uma vez, quando a tela é criada. Aqui preenche o campo de quantidade com o padrão do tipo de campo inicial.

Dropdown que muda outro campo

Trocar o tipo de campo preenche a quantidade sozinho ([2.1](#)). O campo só fica editável se

```
_tipoCampo.qtdConfiguravel.
```

DropDownButtonFormField

Lista de opções; `items` é montado a partir de `TipoCampo.values` (todos os valores do enum).

showDatePicker / showTimePicker

Abrem o calendário e o relógio do Material. `firstDate: DateTime.now()` impede escolher uma data no passado.

Juntando data e hora

```
DateTime(ano, mês, dia, hora, minuto)
```

monta a data final.

CampoDeToque

Data e horário aparecem como campos ([7.2](#)).

8.6 criar_grupo_screen.dart

Já explicado em [8.5](#): tipo de campo e quantidade, seletor de horário, botão de criar.

Diferenças do racha recorrente:

Dia da semana em vez de data

Um dropdown com `DiaSemana.values`. A data de cada rodada é calculada depois.

Horário formatado "HH:mm"

`hour.toString().padLeft(2, '0')` garante dois dígitos ("09:05").

Foto do racha

`CapaDoRacha` + `escolherFotoDoRacha` (7.4).

Localização no mapa

`escolherLocalizacaoNoMapa` abre a tela do mapa e espera o ponto escolhido (8.8).

SwitchListTile "Aberto para novos jogadores"

Um interruptor. A tela bloqueia o envio se estiver ligado sem localização: o racha nunca apareceria na busca por distância.

8.7 editar_racha_screen.dart e editar_grupo_screen.dart

Já explicado em 8.5 e 8.6: são os mesmos formulários.

A diferença é que os valores começam preenchidos com o que já existe, no `initState`, a partir do modelo recebido pela rota (`widget.racha` / `widget.grupo`). Por isso as variáveis são `late` (C2). Editar o grupo vale para as **próximas** rodadas; a rodada aberta se edita pela própria tela da rodada.

8.8 localizacao_picker_screen.dart

`lib/screens/grupo/localizacao_picker_screen.dart`

```
Future<GeoPoint?> escolherLocalizacaoNoMapa(BuildContext context, {GeoPoint? inicial}) {
  return Navigator.of(context).push<GeoPoint>(<
    MaterialPageRoute(builder: (_) => LocalizacaoPickerScreen(inicial: inicial)),
  );
}

// no build:
FlutterMap(
  mapController: _mapController,
  options: MapOptions(
    initialCenter: _selecionado,
    initialZoom: 15,
    onTap: (_, ponto) => setState(() => _selecionado = ponto),
  ),
  children: [
    TileLayer(urlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png', ...),
    MarkerLayer(markers: [Marker(point: _selecionado, child: const Icon(Icons.location_pin))]),
    const RichAttributionWidget(attributions: [TextSourceAttribution('@ OpenStreetMap contributors')]),
  ],
),
```

Navigator.push que devolve valor

Esta tela não tem rota no `go_router`: é aberta com `Navigator.push` e, ao confirmar, fecha com `Navigator.pop(GeoPoint(...))`. Quem abriu recebe o ponto no `await`.

flutter_map + OpenStreetMap

Mapa gratuito. O `TileLayer` baixa as imagens do mapa ($\{z\}$ = zoom, $\{x\}/\{y\}$ = posição). O `MarkerLayer` desenha o alfinete no ponto escolhido.

onTap

Tocar no mapa move o alfinete.

Crédito obrigatório

A licença do OpenStreetMap exige o texto "© OpenStreetMap contributors" visível.

Stack e Positioned

O corpo é um `Stack` (camadas empilhadas): o mapa embaixo e, por cima, os cartões de resultado da busca e de aviso, posicionados com `Positioned`.

Busca de endereço

Campo na `AppBar` (propriedade `bottom`) que chama o `GeocodingService` (3.4). Escolher um resultado move o mapa com `_mapController.move(ponto, 17)`.

GPS ao abrir

Se é um grupo novo, o `initState` já tenta centralizar na posição atual (3.3). Se falhar, mostra um aviso em vez de abrir num lugar aleatório sem explicação.

8.9 rachas_proximos_screen.dart

lib/screens/descoberta/rachas_proximos_screen.dart

Lista os grupos abertos a novos jogadores, perto de quem está olhando.

```
@override
void initState() {
  super.initState();
  _buscarMinhaPosicao();
}

// no build, depois de ter a posição:
final deOutros = grupos.where((g) => g.adminId != meuUid && !g.membrosFixos.contains(meuUid)).toList();
final comLocal = deOutros.where((g) => g.localizacao != null).toList();
final ordenados = comLocal
  .map((g) => (grupo: g, distancia: distanciaKm(minhaPosicao, g.localizacao!)))
  .toList()
  ..sort((a, b) => a.distancia.compareTo(b.distancia));
final proximos = ordenados.where((par) => par.distancia <= _raioKm).toList();
```

Pega a posição ao abrir

No `initState`, com o `LocationService` (3.3). Enquanto isso, a tela mostra carregando; se der erro, mostra a mensagem e um botão "Tentar novamente".

Filtro em etapas

Tira os grupos da própria pessoa → tira os sem ponto no mapa → calcula a distância de cada um (`distanciaKm`, 6.6) → ordena → fica com os dentro do raio.

Records com nome

`(grupo: g, distancia: ...)` junta o grupo e a distância sem criar uma classe; depois se acessa `par.distancia`.

Filtro no celular, não no banco

O Firestore não sabe filtrar por distância sem bibliotecas extras. Como são poucos grupos, o app baixa os abertos e filtra ele mesmo.

ChoiceChip

Os botões 5/10/25/50 km. Trocar o raio só muda `_raioKm` com `setState`; não consulta o banco de novo.

_Vazio

Quando não há nada, explica **por quê** — nenhum grupo aberto, os abertos são seus, nenhum tem mapa, ou o mais próximo está fora do raio (e diz a quantos km). Cada motivo pede uma ação diferente.

_RachaProximoTile

O cartão de cada grupo: foto grande (aqui ela importa: a pessoa quer ver o campo antes de pedir para entrar), nome, distância e o botão. O texto do botão muda com o pedido da pessoa, observado ao vivo por `minhaSolicitacaoProvider`: "Solicitar entrada" → "Pedido pendente" → "Pedido aprovado" ou "Pedido recusado".

8.10 grupo_detalhe_screen.dart

lib/screens/grupo/grupo_detalhe_screen.dart

A tela de um racha recorrente: cabeçalho com foto, local, horário, membros e pedidos de entrada, e embaixo as abas da rodada aberta.

```
final rachaAsync = ref.watch(rachaAtualDoGrupoProvider(grupo.id));
final grupoAtual = ref.watch(grupoPorIdProvider(grupo.id)).valueOrNull ?? grupo;
final isAdmin = grupoAtual.adminId == meuUid;
...
return rachaAsync.when(
  data: (racha) {
    if (racha == null) return comCabecalho(const Text('Nenhuma rodada aberta no momento.'));
    return RachaTabsSection(racha: racha, cabecalho: cabecalho);
  },
  loading: () => comCabecalho(const CircularProgressIndicator()),
  error: (e, _) => comCabecalho(Text('Erro ao carregar rodada: $e')),
);
```

grupoAtual

O grupo chega pela rota (`extra`), mas isso é uma "foto" do momento em que a Home abriu. Observando o grupo ao vivo, a tela mostra na hora um membro novo ou uma foto trocada. Enquanto o ao vivo não chega, usa o recebido (`?? grupo`).

isAdmin decide o que aparece

O admin vê editar/apagar, adicionar e remover membros, a prancheta de anotador e os pedidos de entrada. Quem só participa vê "Sair do grupo".

PopupMenuButton

O menu de três pontinhos. Os itens mudam conforme `isAdmin` ; o valor escolhido passa por um `switch` num enum (`_AcaoGrupo`).

Cabeçalho que rola

O cabeçalho é passado para a `RachaTabsSection` (8.12), que o faz rolar junto e sair de cena, deixando as abas presas no topo.

PARTES DO ARQUIVO

Nome	O que faz
<code>_MembrosFixosSection</code> / <code>_MembroFixoTile</code>	Lista de membros com foto. Para o admin: ícone de prancheta (liga/desliga anotador, 5.6) e X para remover.
<code>_removerMembroFixo</code>	Se a pessoa já confirmou a rodada aberta, pergunta se tira só do grupo ou também da rodada. O diálogo devolve um valor do enum <code>_SaidaDoGrupo</code> .
<code>_mostrarDialogoAdicionarMembro</code>	Busca por nome/email e adiciona. Diálogo com <code>StatefulBuilder</code> (nota em 8.4).
<code>_SolicitacoesSection</code> / <code>_SolicitacaoTile</code>	Pedidos pendentes com a ficha do jogador (nota, posição, rodadas, MVPs — <code>fichaDoJogadorProvider</code>) e botões aprovar/recusar.

<code>_ResumoDoJogador</code>	A linha da ficha. Jogador sem histórico mostra "Jogador novo" em vez de "O,O ★", que pareceria nota ruim.
<code>_RecusadasSection</code>	Pedidos já recusados, com "Reabrir" — o único caminho de volta para quem foi recusado.
<code>_trocarFotoDoRacha</code>	Abre o seletor de foto (7.4) e grava. A capa nova aparece sozinha, porque a tela observa o grupo ao vivo.

8.11 `racha_detalhe_screen.dart`

A tela de uma rodada aberta pela Home (rodada avulsa ou convite). É curta: observa o racha pelo id da rota e mostra a `RachaTabsSection` sem cabeçalho. O admin de racha **avulso** tem um botão de apagar na AppBar; rodada de grupo não pode ser apagada sozinha.

8.12 racha_tabs_section.dart — as abas da rodada

lib/screens/racha/racha_tabs_section.dart (o maior arquivo do app)

As cinco abas de uma rodada, usadas pela tela do grupo e pela tela do racha: **Participantes**, **Convidados**, **Times**, **Estatísticas** e **Próximo racha**.

```
return DefaultTabController(
  length: 5,
  child: cabecalho == null
    ? Column(children: [_abas, Expanded(child: conteudo)])
    : NestedScrollView(
        headerSliverBuilder: (context, _) => [
          SliverToBoxAdapter(child: cabecalho),
          const SliverPersistentHeader(pinned: true, delegate: _AbasFixas(_abas)),
        ],
        body: conteudo,
      ),
);
```

DefaultTabController

Controla qual aba está aberta. O `TabBar` (os títulos) e o `TabBarView` (o conteúdo) conversam através dele. `length` é o número de abas.

Sem cabeçalho

Abas em cima e conteúdo embaixo; o `Expanded` faz o conteúdo ocupar o resto da tela.

Com cabeçalho: NestedScrollView

O cabeçalho rola e sai de cena; as abas ficam **presas no topo** (`pinned: true`). Antes, com muitos membros, o cabeçalho ocupava a tela toda e as abas sumiam.

Slivers

Pecas de uma área rolável. `SliverToBoxAdapter` coloca um widget comum dentro dela; `SliverPersistentHeader` é um pedaço que pode ficar fixo. `_AbasFixas` só informa a altura e dá fundo opaco às abas.

ABA PARTICIPANTES (_PARTICIPANTESTAB)

Agrupado por status

Confirmados, Pendentes e Recusados separados, com uma função local `grupo(titulo, pessoas)` que monta cada bloco.

Botão "Fazer chamada"

Só para admin ou anotador, só a partir do horário do jogo, e só se a rodada não foi finalizada.

_ParticipanteTile

Foto, nome, status, posição e presença (cor pelo resultado). Para a própria pessoa pendente: ✓ e X. Ao confirmar, se ainda não tem posição, abre antes o diálogo de posição — confirmar e escolher posição viraram um passo só — e agenda o lembrete (3.5). Recusar cancela o lembrete.

_mostrarDialogoPosicao

Posição principal (obrigatória) e secundária (opcional, com "Só a principal"). Mostra a formação do racha para a pessoa escolher sabendo onde vai jogar. Devolve `true` se salvou.

_mostrarChamada

Menu com os confirmados e, em cada linha, um `SegmentedButton` com três opções: chegou, atrasou, faltou.

ABA CONVIDADOS (_CONVIDADOSTAB)

Lista de convidados. Para o admin: aprovar, recusar, remover e **oficializar** — um diálogo que busca a conta que o convidado criou e migra o histórico dele (5.7).

ABA TIMES (_TIMESTAB)

```
final totalElegiveis = confirmados.length + aprovados.length;
final minimo = racha.totalVagas * 2;
final suficiente = totalElegiveis >= minimo;
...
FilledButton.icon(
  onPressed: (suficiente && !timesState.isLoading)
    ? () => ref.read(timesControllerProvider.notifier).gerar(racha)
    : null,
  label: Text(jaGerado ? 'Gerar times novamente' : 'Gerar times'),
),
```

Mínimo para gerar

Dois times completos (`totalVagas * 2`). O texto acima do botão diz quantos faltam.

_AvisoEscalacao

Antes de gerar, avisa se ninguém escolheu goleiro ou se há confirmados sem posição. Não bloqueia, só informa.

Depois de gerados

Mostra os dois times (`_TimeSection` , com o goleiro primeiro), o botão "Avaliar jogadores", o troféu para o admin calcular o MVP e o botão "Finalizar racha".

_FinalizarRachaSection

Pede confirmação e chama `finalizar` (5.5). Fica desativado enquanto houver estatística esperando conferência, mostrando quantas faltam.

ABA ESTATÍSTICAS (_ESTATISTICASTAB)

```
final aberta = racha.status != RachaStatus.finalizado;
final conferente = racha.podeFazerChamada(uid);

bool podeConfirmar(EstatisticaModel? e) =>
  aberta && e != null && !e.confirmada &&
  racha.podeConferirEstatistica(uid, e.jogadorId);
...
_EstatisticaParticipanteTile(
  podeEditar: isAdmin || (aberta && (conferente || jogador.userId == uid)),
  podeConfirmar: podeConfirmar(porJogador[jogador.userId]),
),
```

Quem edita

O jogador edita os próprios números; admin e anotadores editam os de todos (inclusive convidados, que não têm login). Depois de finalizada, só o admin.

Função dentro do build

`podeConfirmar` é uma função local, usada em cada linha.

_ResumoConferencia

O cartão no topo: quantas estão aguardando conferência e, para quem confere, o botão "Confirmar todas (N)".

_EstatisticaTile

Gols, assistências e cartões; "Aguardando conferência" em amarelo ou "Conferido" em verde; ✓ para confirmar e o lápis para editar.

_mostrarDialogoEstatistica

Quatro campos numéricos. `FilteringTextInputFormatter.digitsOnly` só deixa digitar números. Se quem lança não pode conferir, avisa que vai ficar pendente.

ABA PRÓXIMO RACHA (`_PROXIMORACHATAB`)

Data, local e tipo de campo (`InfoTile`). Para o admin: "Compartilhar código" (copia o id do racha com `Clipboard.setData`) e "Editar racha". Para quem foi convidado e ainda não respondeu: um cartão com "Confirmar presença" e "Recusar".

8.13 convidar_jogador_screen.dart

Já explicado em **8.2**: formulário com idade e peso lado a lado. Em **4.1**: a busca por nome ou email.

Duas partes na mesma tela:

Jogador cadastrado

Busca e um botão "Convidar" em cada resultado (`for (final user in _resultados) ListTile(...)` dentro da lista, **C11**). O convite nasce pendente.

Convidado sem cadastro

Nome, idade e peso aproximados e posições. Fica pendente até o admin aprovar. Ao adicionar, os campos são limpos com `.clear()` para cadastrar o próximo.

8.14 avaliacao_screen.dart

```
final Map<String, double> _notasCompanheiros = {};
AlvoAvaliacao? _adversarioEscolhido;
double? _notaAdversario;
...
final contextoAsync = ref.watch(contextoAvaliacaoProvider((rachaId: widget.rachaId, uid: uid)));
...
final formCompleto = companheiros.every((c) => _notasCompanheiros[c.id] != null) &&
    (adversarios.isEmpty || (_adversarioEscolhido != null && _notaAdversario != null));
```

A regra

Cada jogador dá nota a **todos os companheiros do próprio time** e escolhe **um adversário**.

Map de notas

`_notasCompanheiros` guarda a nota de cada companheiro pelo id. Tocar numa estrela chama `setState` e grava ali.

Family com record

O provider recebe dois valores juntos: `(rachaId: ..., uid: ...)`.

formCompleto

`every` = "todos cumprem a condição". O botão só habilita com todas as notas dadas.

Estados da tela

Já avaliou → agradece. Sem time (não jogou ou times não gerados) → explica. Senão → o formulário.

_NotaSelector

Cinco `IconButton`s de estrela, criados com `for (var i = 1; i <= 5; i++)`; cheia se a nota for maior ou igual a `i`.

_AdversarioChip

`ChoiceChip` com foto: só um pode ficar selecionado.

ref.invalidate

Depois de enviar, manda o provider buscar de novo, e a tela passa a mostrar "Você já avaliou".

8.15 **grupo_ranking_screen.dart e historico_grupo_screen.dart**

Já explicado em [C7](#): `.when` com carregando/erro/dado.

Ranking

Observa `rankingDoGrupoProvider` ([5.1](#)) e mostra uma lista com `ListView.separated` (uma linha divisória entre os itens). A foto tem um selo com a posição; o primeiro lugar ganha selo dourado (`Stack` + `Positioned`). Quem nunca foi avaliado mostra "—" em vez de "0.0", que pareceria a pior nota.

Histórico

Rodadas finalizadas do grupo, com data por extenso (`DateFormat("EEEE, dd/MM/yyyy 'às' HH:mm", 'pt_BR')`) e o MVP de cada uma. Tocar abre a rodada.

Perguntas que o professor pode fazer

Respostas curtas, com o link para a explicação completa.

Onde o app usa a câmera?

No `FotoPerfilPicker`, com `ImagePicker().pickImage(source: ImageSource.camera)`, no cadastro e na edição de perfil. → [7.3](#)

Como a tela sabe que alguém fez login e vai para a Home?

O router observa o stream de login do Firebase. Quando muda, o `redirect` roda de novo e manda para `/home`. A tela de login não navega. → [1.2](#)

Qual a diferença entre `ref.watch` e `ref.read`?

`watch` lê e fica observando, redesenhando a tela quando muda (usado no `build`). `read` lê uma vez (usado em botões). → [C6](#)

Para que serve o `dispose`?

Libera os `TextEditingController`s quando a tela fecha, para não ficar ocupando memória. → [8.1](#)

O que é `fromMap` e `toMap`?

Conversões entre o objeto do Dart e o mapa que o Firestore grava. `toMap` para salvar, `fromMap` para ler. → [2.2](#)

Por que separar em `services`, `repositories` e `controllers`?

Cada camada tem uma responsabilidade. Só `services` e `repositories` conhecem o Firebase; a lógica de times e ranking fica em `core/` sem Firebase, e por isso dá para testá-la sem internet. → [0.2](#)

Como os times são balanceados?

Um goleiro por time; mesma quantidade em defesa, meio e ataque, pela posição principal; depois troca jogadores do mesmo setor até os times ficarem parecidos em nota (peso 1,0), gols (0,6), idade (0,4) e peso corporal (0,15). → [6.3](#)

E se faltar meia e sobrar atacante?

Primeiro entra quem não escolheu posição. Depois, atacantes que marcaram meia como posição secundária descem, começando pelos de menor nota. Ninguém joga fora das duas posições que escolheu. → [6.2](#)

Que nota tem um jogador novo ou convidado?

3,0, o meio da escala, para não puxar o equilíbrio para nenhum lado. Nos gols, a média do grupo. → [6.1](#)

Como é calculado o ranking?

Sempre do zero, a partir de todas as avaliações, estatísticas conferidas e MVPs do jogador, numa função só (`agregarRanking`), usada pelo ranking geral e pelo do grupo. Ordem: mais MVPs, depois maior média. → [6.4](#)

Como é escolhido o MVP?

Maior média de notas na rodada; empate, quem recebeu mais avaliações. Convidado nunca é MVP. → [6.5](#)

Um jogador pode inventar os próprios gols?

Pode lançar, mas fica "aguardando conferência" e não conta no ranking até o admin ou um anotador confirmar. O anotador não confere os próprios números, e o racha só finaliza com tudo conferido. A regra também está no `firestore.rules`. → [5.11](#)

O que acontece quando o admin finaliza uma rodada de grupo?

Num único batch: a rodada vira "finalizada", nasce a próxima (data calculada pelo dia da semana) e o admin e os membros fixos são convidados. → [5.5](#)

O que é um WriteBatch e por que usar?

Várias gravações enviadas juntas: ou todas acontecem, ou nenhuma. Evita estados quebrados, como um racha sem participantes ou times atribuídos pela metade. → [C10](#)

Como funciona a busca por "Rachas Próximos"?

Pega a posição pelo GPS, baixa os grupos abertos e calcula a distância de cada um com a fórmula de haversine, filtrando pelo raio escolhido. → [8.9](#)

Por que a foto é guardada em base64?

O Firebase Storage exige plano pago. A foto é comprimida, convertida em texto e gravada no documento. O `StorageService` isola isso para trocar depois sem mexer em quem chama. → [3.2](#)

O que é o AsyncValue.guard?

Executa a ação e captura qualquer erro, guardando no estado do controller em vez de deixar o app quebrar. A tela lê `isLoading` e `error` desse estado. → [C8](#)

Por que tem "if (!mounted) return" depois de await?

Durante a espera a pessoa pode ter saído da tela; usar o `context` de uma tela fechada quebra o app. → [C12](#)